

TestiQra: An AI-Powered Testimonial Collection Web Application

DISSERTATION

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

Master of Computer Applications

By

Sonali Varshney

Roll No.: 24CAMSA162



Under the Supervision of

DR. SEHBA MASOOD

DEPARTMENT OF COMPUTER SCIENCE

ALIGARH MUSLIM UNIVERSITY

ALIGARH (INDIA)

2026

Certificate

DEPARTMENT OF COMPUTER SCIENCE ALIGARH MUSLIM UNIVERSITY, ALIGARH

This is to certify that the dissertation entitled “**TestiQra — An AI-Powered Testimonial Collection Web Application**” submitted by **Sonali Varshney**, Roll No. 24CAMSA162, in partial fulfilment of the requirements for the award of the degree of *Master of Computer Science and Applications* to the **Aligarh Muslim University, Aligarh**, is a record of bonafide work carried out by the candidate under my supervision and guidance.

To the best of my knowledge and belief, the work has not been submitted for the award of any other degree or diploma in this or any other university or institution.

Supervisor

Head of Department

Dr. Sehba Masood

Assistant Professor

Department of Computer Science

Aligarh Muslim University, Aligarh

Dr. Arman Rasool Faridi

Head, Department of Computer Science

Aligarh Muslim University, Aligarh

Date: _____

Place: Aligarh

Acknowledgement

First and foremost, I express my deepest gratitude to the Almighty for bestowing upon me the wisdom, perseverance, and creative vision necessary to conceive and complete this dissertation. It is a profound privilege to acknowledge everyone whose support, guidance, and encouragement made this work possible.

I owe an immense debt of gratitude to my esteemed supervisor, **Dr. Sehba Masood**, Assistant Professor, Department of Computer Science, Aligarh Muslim University, Aligarh. Her scholarly mentorship extended far beyond the technical aspects of system design; she consistently challenged me to think more rigorously about software architecture, user experience, and the ethical implications of deploying AI in commercial settings. Her door was always open for discussion, and her constructive critiques transformed early drafts into the coherent academic work presented here.

I am deeply thankful to **Dr. Arman Rasool Faridi**, Chairman of the Department of Computer Science, AMU, for fostering an intellectually vibrant departmental environment and for ensuring that students have access to the computational infrastructure and academic resources necessary for advanced project work. I also extend my appreciation to all the faculty members of the department whose courses in advanced database management, web technologies, artificial intelligence, and software engineering provided the theoretical underpinning for this project.

Finally, no acknowledgement would be complete without recognising the unconditional love and moral support of my family. Their belief in my capabilities, even during periods of uncertainty, remains the most enduring source of strength I carry.

Sonali Varshney

24CAMSA

Enrolment No. GL3905

Contents

Certificate	i
Acknowledgement	ii
Abstract	1
1 Introduction	2
1.1 Introduction	2
1.2 Literature Survey	3
1.2.1 Previous Related Work	3
1.2.2 Company Profile	4
1.3 Present State of the Art and Its Shortcomings	4
1.4 Realisation of the Problem	5
1.5 Introduction to the Problem and Work Undertaken	6
1.6 Broad Outline of the Work (Chapter-wise)	7
2 Problem Formulation	8
2.1 Problem Definition and Detailed Description	8
2.2 Various Aspects of the Problem	9
2.3 Present System — Critical View	10
2.4 Scientific Novelty and Need of the Work	11
2.5 Proposed System and Method of Solution	11
3 System Analysis and Design	13
3.1 Requirement Specification	13
3.1.1 Functional Requirements	13
3.1.2 Non-Functional Requirements	14
3.2 Information Collection	14
3.3 System Architecture	15
3.4 Analysis and Development of the Actual Solution	16
3.4.1 Database Schema	16
3.5 Description of Various Modules	18
3.5.1 Module 1 — User Authentication	18
3.5.2 Module 2 — Space Management	18

3.5.3	Module 3 — Testimonial Collection	19
3.5.4	Module 4 — AI Features	19
3.5.5	Module 5 — Wall of Love	20
3.5.6	Module 6 — Email Dispatch	20
3.6	System Development Tools	20
3.7	Choice of Language	21
3.8	Choice of System for Implementation	22
3.9	REST API Specification	23
4	System Implementation	25
4.1	Hardware Requirements	25
4.2	Software Requirements	25
4.3	Input Requirements	26
4.4	Key Code Implementations	27
4.4.1	JWT Authentication Middleware	27
4.4.2	Groq AI Integration	27
4.4.3	Multi-Layer Spam Prevention Pipeline	28
4.5	Operation Manual	29
4.5.1	For Business Owners (Dashboard Users)	29
4.5.2	For Reviewers (Submission Form Users)	30
4.6	Output Reports and Artefacts	30
5	Testing	32
5.1	API Route Testing Methodology	32
5.1.1	User Authentication Routes	32
5.1.2	Space Management Routes	32
5.1.3	Space Fetching Routes	33
5.1.4	Testimonial Submission and Retrieval Routes	33
5.1.5	AI Inference Route	33
5.1.6	Email Reply Route	33
6	Results and Discussion	34
6.1	Brief Evaluation of the System	34
6.2	Interpretation of Results	34
6.2.1	Performance Benchmarks	34
6.2.2	Route-Level Functional Test Results	35
6.3	Application Areas and Scope of the Work	36
6.3.1	SaaS and Software Products	36
6.3.2	E-Commerce	37
6.3.3	Coaching and Consulting	37
6.3.4	Educational Institutions	37

6.3.5	Non-Profit Organisations	38
6.3.6	Recruitment and Employer Branding	38
7	Conclusion	39
7.1	Advantages and Special Features of the System	39
7.2	Limitations	40
7.3	Future Extension	41
7.3.1	Short-Term Extensions (0–6 Months)	41
7.3.2	Medium-Term Extensions (6–18 Months)	41
7.3.3	Long-Term Extensions (18+ Months)	42
	Bibliography	43
	Appendix	45
	A1: Architectural Diagram	45
	A2: Entity-Relationship Diagram	45
	A4: System Flow Diagram	45
	A5: Program Codes	45
	A6: Application Screenshots	45

Abstract

In today's digitally driven economy, authentic customer testimonials serve as one of the most powerful trust-building mechanisms available to businesses of every scale. Contemporary consumer-behaviour research indicates that approximately 79% of online shoppers place as much trust in digital reviews as they do in personal recommendations from friends or family [1]. Furthermore, organisations that proactively curate and publish testimonial collections experience measurable conversion rate improvements of up to 34% compared to competitors who rely solely on product descriptions and marketing copy. Despite this well-documented importance, the end-to-end process of collecting, moderating, analysing, and embedding testimonials remains a largely manual and fragmented exercise for the vast majority of small and medium-sized businesses.

This dissertation presents **TestiQra**, a full-stack, AI-augmented web application purpose-built to streamline every stage of the testimonial life cycle — from initial campaign setup through submission, analysis, personalised response dispatch, and public gallery publication. TestiQra enables business operators to create and configure shareable *Spaces* — branded, publicly accessible landing pages through which customers can submit multi-modal testimonials in both text and video formats. The platform integrates a large language model (LLM) via the Groq API (Llama 3.3-70B) to deliver five distinct AI-powered capabilities: (i) automated content generation for Space configuration, including headers, welcome messages, and targeted questions; (ii) holistic sentiment analysis and executive summarisation of entire testimonial corpora; (iii) AI-powered improvement coaching that maps recurring complaints to actionable remediation priorities; (iv) intelligent testimonial curation recommendations for the Wall of Love gallery; and (v) AI-drafted, contextually personalised reply emails for individual reviewers.

A dedicated *Wall of Love* feature allows businesses to seamlessly embed curated testimonial galleries directly on their own websites using four distinct display layouts — Marquee, Spotlight, Cascade, and Grid — and five visual themes. The underlying architecture follows a modern three-tier client-server model: **React.js** (Vite) powers the front-end single-page application, a **Node.js/Express.js** REST API orchestrates all business logic, and **PostgreSQL** managed via **Prisma ORM** provides type-safe, relational data persistence. Binary media assets including images and video testimonials are stored on **Cloudinary's** globally distributed CDN. The authentication layer supports both password-based and OAuth 2.0 (Google Sign-In) flows, ensuring accessibility for a broad user base.

Chapter 1

Introduction

1.1 Introduction

The rapid proliferation of e-commerce platforms, Software-as-a-Service (SaaS) products, and on-demand digital services has fundamentally transformed the way businesses establish trust with prospective customers. In an online marketplace saturated with competing products and commoditised value propositions, authentic social proof — in the form of customer testimonials, detailed reviews, and verifiable success stories — has emerged as one of the single most influential determinants in purchase decision-making. The transition from physical shopping environments, where peer recommendations were exchanged in person, to digital storefronts, where anonymous reviewers carry enormous persuasive weight, has created both an opportunity and a challenge for business operators.

Research by BrightLocal (2023) quantifies this phenomenon: 79% of consumers report placing as much trust in online reviews as in personal recommendations from people they know [1]. This statistic is not merely anecdotal; it reflects a deep and measurable shift in consumer psychology enabled by the transparency of digital commerce. Organisations that proactively collect and showcase customer testimonials consistently report conversion rate improvements of up to 34% compared to those that do not engage in any form of social proof marketing. For a startup or small business operating with limited marketing budgets, this differential can represent the margin between commercial viability and obscurity.

Simultaneously, the democratisation of cloud computing has made enterprise-grade web application capabilities accessible to individual developers and small teams. Cloud platforms now offer database hosting, media storage, global content delivery, and AI inference — all on generous free tiers — allowing a single engineer or a small research group to deploy a production-quality application to a global audience at essentially zero cost. This convergence of market need and accessible technology forms the backdrop against which TestiQra was conceived.

TestiQra is an intelligent testimonial management platform designed to address a clearly defined gap: no existing commercial or open-source solution unifies the full testimonial lifecycle — from campaign creation through AI-assisted insight extraction to embeddable publication — under a single, coherent interface. By combining modern full-stack web engineering with the inference power of large language models, TestiQra aims to make high-quality testimonial management accessible to businesses that previously lacked the time, expertise, or budget to pursue it systematically.

This dissertation documents the complete design, implementation, evaluation, and future

roadmap of TestiQra, developed as the MCA IV Semester capstone project at the Department of Computer Science, Aligarh Muslim University, Aligarh.

1.2 Literature Survey

1.2.1 Previous Related Work

The commercial landscape for testimonial and review management tools has evolved significantly over the past decade, though it remains fragmented and lacking in intelligent automation.

Testimonial.to is widely credited with popularising the concept of a dedicated “Space” — a uniquely branded, shareable URL through which a business’s customers can submit text and video testimonials without requiring registration. The platform introduced the notion of the “Wall of Love” as an embeddable widget. However, the platform’s scope is limited to collection and display; it offers no analytical capabilities beyond aggregate star ratings and provides no AI-assisted content generation or reply workflow.

Senja extended the *Testimonial.to* model by adding robust import pipelines from external review aggregators such as G2, Trustpilot, and Capterra, allowing businesses to centralise reviews from multiple sources. *Senja* supports richer display widgets and has a more polished user interface. Nevertheless, like its predecessor, it remains analytically passive — testimonials are stored and displayed but not computationally analysed for sentiment, patterns, or actionable insights.

Boast.io targets the enterprise segment with a focus almost exclusively on video testimonials. It supports guided video recording workflows, request automation via email campaigns, and basic analytics dashboards. The platform’s strength in video production quality comes at the expense of accessibility; its pricing structure places it beyond the reach of small businesses, and it does not support text testimonials in any meaningful capacity.

Google Business Reviews remains the dominant testimonial platform by sheer volume, particularly for local and physical businesses. However, it is fundamentally a review aggregation tool rather than a collection management platform. Businesses have no control over the collection form, cannot embed a curated widget on their own websites, and receive no analytical intelligence from the platform.

In the academic literature, the foundational work on automated opinion analysis was laid by Pang and Lee (2008), whose comprehensive survey of opinion mining and sentiment analysis techniques established the taxonomic framework still referenced today. Their distinction between document-level, sentence-level, and aspect-level sentiment analysis remains the conceptual basis for modern review analytics systems. Turney (2002) demonstrated that unsupervised learning methods could achieve 74% accuracy on sentiment classification of reviews, establishing that automated approaches could approximate human judgment at scale.

The transformer revolution fundamentally altered the state of the art. Devlin et al. (2019)

introduced BERT (Bidirectional Encoder Representations from Transformers), demonstrating that pre-trained deep language models fine-tuned on downstream tasks achieved near-human performance on sentiment classification benchmarks. This research established the theoretical justification for using LLM APIs — rather than custom-trained models — for testimonial analysis, since modern hosted models like Llama 3.3-70B encode vastly more contextual understanding than task-specific classifiers trained on small labelled corpora.

More recently, the emergence of instruction-tuned LLMs accessible via low-latency inference APIs (such as Groq’s Llama 3.3-70B) has made it architecturally practical to integrate sophisticated language understanding directly into web application request-response cycles without the latency penalties that previously made LLM integration impractical for real-time user-facing applications.

Prompt engineering as a discipline has also matured rapidly. Brown et al. (2020) demonstrated the few-shot learning capabilities of GPT-3, establishing that carefully constructed prompts could elicit structured, reliable outputs from large language models without fine-tuning. Subsequent work on instruction tuning and Reinforcement Learning from Human Feedback (RLHF) produced models that respond reliably to explicit output format constraints, making JSON-mode completions a practical basis for building parseable, programmatically consumable AI features within web applications.

1.2.2 Company Profile

TestiQra is an independent, project developed entirely within the academic context of the MCA IV Semester programme at the Department of Computer Science, Aligarh Muslim University, Aligarh, under the supervision of Dr. Sehba Masood. The project has no commercial affiliation, corporate sponsorship, or revenue model. All external services, Groq AI inference, Cloudinary media storage, and Google OAuth — are accessed exclusively via their publicly available free-tier API plans during development and evaluation. The codebase is structured to facilitate deployment on the free tiers of Render (Node.js hosting), Vercel (React hosting), and Neon (serverless PostgreSQL), reflecting the project’s commitment to accessibility and reproducibility by other researchers and practitioners.

1.3 Present State of the Art and Its Shortcomings

A careful survey of commercially available testimonial management platforms reveals five consistently recurring limitations that collectively constrain the platform’s utility, particularly for non-technical business operators:

- **Absence of AI-assisted content creation:** Business owners are required to independently formulate collection questions, write persuasive header copy, and craft welcome messages — tasks that demand marketing expertise and considerable time investment. Platforms provide no guidance, no templates that adapt to business type or target audience, and no

generation capability. This barrier causes many businesses to deploy generic, ineffective collection forms that yield low submission rates and superficial responses.

- **No automated insight extraction:** Platforms universally present testimonials as raw text strings, leaving the analytical burden entirely on the business owner. With even modest testimonial volumes (30–50 submissions), manually reading, categorising, and extracting themes from testimonial corpora becomes a multi-hour exercise. No commercial platform currently offers automated sentiment classification, thematic clustering, or AI-generated executive summaries of testimonial collections.
- **Absence of an integrated reply workflow:** When a business owner wishes to acknowledge a reviewer or respond to a concern raised in a testimonial, they must manually copy the reviewer’s contact information, open a separate email client, compose a personalised response from scratch, and send it — all outside the testimonial management platform. This context switch introduces friction that results in the vast majority of reviewers never receiving any acknowledgement, which in turn reduces the likelihood of positive word-of-mouth amplification.
- **Limited and inflexible embedding customisation:** Embeddable widgets offered by existing platforms typically provide two or three fixed layout options with minimal theming controls. For businesses with established visual identities, these widgets often clash visually with their website’s design language, leading to inconsistent brand experiences. No platform offers real-time live preview of widget customisation before embedding.
- **Weak and single-layer spam protection:** Most platforms rely on a single CAPTCHA implementation as their sole spam prevention mechanism. CAPTCHA is increasingly bypassable by modern bot frameworks and creates unnecessary friction for genuine human submitters. No surveyed platform implements layered defences that combine multiple independent prevention mechanisms.

1.4 Realisation of the Problem

The core insight driving this project is architectural rather than feature-specific: testimonial management is not a single discrete task but rather a *multi-stage workflow* comprising five sequentially dependent stages — campaign setup, testimonial collection, corpus analysis, personalised response dispatch, and public gallery publication. Each of these stages currently demands either significant manual effort, specialised expertise, or a disruptive context switch to a different tool. No single platform addresses all five stages coherently, and none applies AI to reduce the cognitive burden at each transition point.

For small business owners — who frequently serve as their own marketing manager, customer service representative, and web developer simultaneously — this workflow fragmentation means that testimonial collection is treated as an afterthought rather than a strategic asset. The consequence is that potentially transformative social proof is either collected haphazardly

(using general-purpose Google Forms with no collection intelligence), stored passively (with no analysis or follow-up), or simply not collected at all due to the perceived complexity of setting up a proper collection system.

The realisation that unifying this workflow under a single intelligent platform could unlock significant value for this underserved segment — without requiring any increase in technical skill from the end user — became the central motivation for TestiQra.

1.5 Introduction to the Problem and Work Undertaken

This dissertation undertakes the end-to-end design, implementation, and empirical evaluation of **TestiQra**, a unified platform that systematically addresses all five identified shortcomings described in Section 1.3. The scope of work undertaken encompasses the following principal activities:

1. **Full-stack web application engineering:** Design and implementation of a React.js single-page application front-end (using Vite as the build tool), a Node.js/Express.js RESTful API back-end with JWT-protected management endpoints, and a PostgreSQL relational database managed via Prisma ORM. The application supports dark and light themes, is fully responsive across desktop and mobile viewports, and is structured to support independent horizontal scaling of front-end and back-end tiers.
2. **AI feature integration:** Design of five structured prompt templates and integration of the Groq Llama 3.3-70B inference API for the following capabilities: Space content generation, sentiment summarisation, improvement coaching, testimonial curation recommendation, and personalised reply drafting. All AI interactions use JSON-mode completions to guarantee parseable, programmatically consumable outputs.
3. **Wall of Love widget design:** Implementation of a standalone, unauthenticated display page supporting four layout modes (Marquee, Spotlight, Cascade, Grid), five visual themes, six font selections, and real-time customisation preview. Generation of both iframe-based and script-based embed code snippets.
4. **Spam prevention pipeline:** Design and implementation of a four-layer submission defence combining honeypot field detection, IP-based rate limiting, email-based rate limiting, and content pattern filtering — operating without any third-party CAPTCHA dependency.
5. **Performance evaluation:** Structured empirical evaluation using manual timing studies, controlled bot submission tests, and API response latency benchmarking against the non-functional requirements defined in the system design phase.

1.6 Broad Outline of the Work (Chapter-wise)

The dissertation is organised into six chapters and five appendices as follows:

Chapter 2 — Problem Formulation: Provides a formal and comprehensive definition of the problem, including a detailed competitive analysis of existing platforms, a multi-dimensional characterisation of the problem space, and a specification of the proposed system’s method of solution.

Chapter 3 — System Analysis and Design: Presents the complete system specification including functional and non-functional requirements, information gathering methodology, detailed architectural description of the three-tier system, database schema analysis, module-level design, technology stack justification, and REST API specification.

Chapter 4 — System Implementation: Documents the practical implementation of the system, covering hardware and software environment requirements, detailed explanations of key implementation decisions for authentication, AI integration, and spam prevention, a complete operational manual for both business owners and reviewers, and a description of all system output artefacts.

Chapter 5 — Result and Discussion: Presents empirical evaluation results with full methodological justification, including performance benchmarks, space setup time comparison studies, spam prevention effectiveness analysis, and discussion of application domains across multiple industry verticals.

Chapter 6 — Conclusion: Summarises the scientific contributions of the work, discusses advantages and limitations of the implemented system, and presents a time-phased roadmap for future extensions.

Appendices: Contains the architectural diagram, entity-relationship diagram, use case diagram, system flow diagram, complete annotated program code listings, and application screenshots.

Chapter 2

Problem Formulation

2.1 Problem Definition and Detailed Description

The central problem addressed by this dissertation can be formally stated as follows:

Small and medium-sized businesses lack access to an affordable, unified, and intelligent platform capable of assisting them in (a) establishing high-quality testimonial collection campaigns with minimal configuration effort, (b) automatically extracting actionable behavioural and sentiment insights from collected testimonials using large language model technology, (c) responding to individual reviewers in a personalised and timely manner without disrupting their primary workflow, and (d) presenting curated testimonial galleries in visually compelling, brand-consistent formats on their own websites.

This overarching problem decomposes into three compounding sub-problems, each of which represents a distinct barrier to effective testimonial management practice:

1. **Collection Friction:** The act of soliciting meaningful testimonials from customers requires deliberate effort from both the requesting business and the responding customer. On the business side, crafting psychologically effective collection questions, designing an appealing submission interface, and managing follow-up communication demands marketing expertise and time that most small business owners do not possess. On the customer side, open-ended text boxes with no guiding structure produce either superficial, generic responses ("Great product, highly recommend!") or no response at all, as customers are uncertain about what level of detail is expected. The compound effect of these barriers is chronically low testimonial response rates and low testimonial quality, even in businesses with genuinely satisfied customer bases.
2. **Analysis Overhead:** Even when testimonials are successfully collected in moderate volumes (30 to 100 submissions), extracting strategic value from the corpus requires substantial analytical effort. A business owner must read each submission individually, mentally categorise recurring themes, identify which positives to amplify and which complaints to address, and form an overall impression of customer sentiment — all without any computational assistance. Studies in cognitive load management suggest that human readers can reliably extract thematic patterns from no more than 15–20 text samples before diminishing returns set in; beyond that threshold, important signals are frequently missed. For businesses receiving hundreds of testimonials annually, this analysis overhead renders the raw testimonial data practically unusable as strategic input.

-
3. **Publication Complexity:** Even after testimonials have been collected and mentally processed, making them publicly visible on a business’s own website requires technical integration work. Most website builders and content management systems (WordPress, Webflow, Squarespace) do not include native testimonial gallery components. A business owner must either embed a third-party widget with limited visual customisation options, manually insert testimonials as static HTML that requires developer intervention for updates, or pay for a separate review display plugin. Each of these paths introduces friction, cost, or technical dependency that deters many businesses from maintaining up-to-date testimonial galleries.

A secondary but operationally critical concern is **data integrity**: any public-facing web form constitutes an attack surface for automated spam bots, which routinely harvest form endpoints from publicly indexed pages and flood them with irrelevant, malicious, or promotional submissions. A testimonial corpus contaminated with spam cannot be reliably used for analysis, trusted for display, or preserved as an accurate record of genuine customer experience. Spam prevention is therefore not a peripheral feature but a foundational requirement for any credible testimonial management system.

2.2 Various Aspects of the Problem

A rigorous characterisation of the testimonial management problem reveals that it spans four distinct and partially independent dimensions, each demanding a different class of solution:

- **Technical dimension:** The system must handle multi-modal inputs spanning plain text, uploaded images, and recorded or uploaded video files — each with distinct storage, processing, and delivery requirements. It must integrate reliably with external AI inference APIs, media CDN platforms, SMTP email services, and OAuth identity providers, each with their own rate limits, authentication schemes, and failure modes. It must sustain acceptable response times under realistic concurrent user loads and remain deployable on cost-efficient, freely available infrastructure tiers.
- **User experience dimension:** The platform serves two fundamentally different user populations with radically different technical profiles and interaction goals. The *reviewer* (customer submitting a testimonial) is typically a non-technical user accessing the platform once, with no prior context, and must be able to complete the submission form without instructions, confusion, or friction. The *business owner* (dashboard user) is a semi-technical user who may interact with the platform multiple times per week, requiring an information-dense management interface that still remains navigable without formal training.
- **Security dimension:** The submission endpoint is a public-facing, unauthenticated service by design — it must be accessible to customers without login. This openness creates a direct conflict with security objectives: the endpoint must simultaneously remain frictionless

for legitimate human users and resistant to automated abuse. Traditional CAPTCHA solutions partially address automated submission but introduce measurable UX friction and are increasingly bypassed by advanced bot frameworks. A more sophisticated, multi-layered defence strategy is required.

- **AI reliability dimension:** Deploying a large language model within an automated web application request cycle introduces reliability constraints that do not apply to LLM use in conversational settings. The model’s outputs must be structurally consistent (parseable JSON, not free-form prose) across hundreds of calls with varying input content; the prompt must be engineered to prevent the model from defaulting to markdown formatting, preamble text, or structural variations that would break the application’s JSON parser. Inference latency must remain within the acceptable range for synchronous HTTP responses (under 4 seconds), and the system must handle API downtime gracefully without crashing or returning unintelligible error messages to users.

2.3 Present System — Critical View

Table 2.1 presents a detailed feature-level comparative analysis of the four most significant testimonial management platforms currently available, measured against TestiQra’s feature set. The comparison encompasses nine capability dimensions selected on the basis of their direct relevance to the problem dimensions identified in Section 2.2.

Table 2.1: Comparative Analysis of Testimonial Management Platforms

Feature	Testimonial.to	Senja	Boast.io	Google	TestiQra
Text Collection	✓	✓	✓	✓	✓
Video Collection	✓	✓	✓	×	✓
AI Content Generation	×	×	×	×	✓
AI Sentiment Analysis	×	×	×	×	✓
AI Reply Drafting	×	×	×	×	✓
Embeddable Widget	✓	✓	×	×	✓
Multi-layer Spam Filter	×	×	×	×	✓
Open Source	×	×	×	×	✓
Free Tier	Limited	Limited	Limited	✓	✓

The table reveals a clear pattern: existing platforms compete primarily on the quality of their collection and display mechanisms, treating analysis, response, and security as secondary concerns. The three AI-related capabilities — content generation, sentiment analysis, and reply drafting — are absent from every surveyed competitor. This absence is not incidental; it reflects a product design philosophy that treats testimonials as artefacts to be stored and displayed rather than as data to be computationally understood and acted upon. TestiQra’s design philosophy inverts this priority ordering, treating AI-assisted intelligence as a first-class feature alongside

collection and display.

2.4 Scientific Novelty and Need of the Work

The primary scientific novelty of this work lies in the **structured, end-to-end integration of a generative large language model into the complete testimonial management lifecycle** — not at a single stage in isolation, but across all five workflow stages: campaign setup, active collection, corpus analysis, response generation, and gallery publication. This holistic approach distinguishes TestiQra from the existing academic literature, which has overwhelmingly focused on sentiment classification as a standalone natural language processing task applied to third-party datasets, without addressing the workflow context in which such analysis would be practically deployed.

A secondary scientific contribution is the demonstration that sophisticated AI capabilities can be delivered within a production-quality, open-source web application built exclusively on freely available infrastructure. This represents a meaningful counter-narrative to the prevailing assumption that AI-augmented business tools require enterprise budgets or proprietary platform lock-in.

The practical need for this work is reinforced by the accessibility gap observed in the market: enterprise-grade customer feedback analytics platforms such as Qualtrics and Medallia offer sophisticated text analytics and sentiment dashboards but are priced at thousands of dollars per month per seat, placing them entirely beyond the reach of small and medium-sized businesses. TestiQra demonstrates that the core analytical capabilities of such platforms can be replicated using publicly available LLM APIs at effectively zero marginal cost per analysis request.

2.5 Proposed System and Method of Solution

TestiQra addresses all identified gaps through a unified platform architected on the following five key design decisions:

- **Space-centric data model:** Each distinct testimonial collection campaign is encapsulated in a discrete database entity called a “Space,” which carries its own branding configuration, structured set of collection questions, testimonial corpus, and display preferences. This encapsulation allows a single business owner to manage multiple parallel campaigns — for different products, services, or customer segments — within a single account without data cross-contamination.
- **Groq Llama 3.3-70B integration with JSON-mode prompting:** All five AI-powered features are routed through a single, centralised Express router endpoint that proxies structured requests to the Groq inference API. Each feature uses a carefully engineered prompt template that explicitly specifies the required JSON output schema, constrains the model’s temperature for consistency, and includes a system instruction mandating JSON-

only responses without markdown formatting. This architecture ensures that all AI outputs are directly parseable by the application's response handler without any post-processing.

- **In-app email dispatch via Nodemailer/Gmail SMTP:** Rather than requiring business owners to copy reviewer contact details and compose responses in a separate email client, TestiQra integrates a JWT-protected email dispatch endpoint that allows AI-drafted reply emails to be sent directly from the dashboard with a single action. This eliminates the principal context switch that prevents most businesses from responding to reviewers at all.
- **Customisable Wall of Love with real-time preview:** The gallery publication feature provides five visual themes (Light, Dark, System, Neon, and Minimal), four layout modes, six font families, and real-time slider controls for spacing and size parameters. A live preview renders the gallery with the current configuration before the business owner generates the embed snippet, eliminating trial-and-error integration cycles.
- **Four-layer spam prevention without CAPTCHA:** The submission endpoint implements four independently functioning defence layers, honeypot field detection, IP-based rate limiting with progressive blocking, email-based rate limiting, and content pattern filtering, that operate cumulatively and without any third-party CAPTCHA dependency, preserving submission form usability while substantially reducing bot success rates.

Chapter 3

System Analysis and Design

3.1 Requirement Specification

3.1.1 Functional Requirements

The following functional requirements were derived from the problem analysis conducted in Chapter 2 and refined through competitive platform evaluation and informal user interviews described in Section 3.2.

1. The system shall allow a business owner to create a new account using either an email/password combination or a Google OAuth 2.0 sign-in, and shall subsequently allow sign-in, profile management, and account deletion.
2. The system shall allow authenticated business owners to create, read, update, and delete *Spaces*, each of which encapsulates a complete testimonial collection campaign with its own branding, questions, and corpus.
3. Each Space shall support a publicly accessible, unauthenticated submission form that accepts both text-based testimonials (with optional image attachment) and video testimonials (uploaded file or browser-recorded).
4. The system shall expose an AI-powered content generation capability that, given basic company metadata, automatically populates all Space configuration fields (header, welcome message, and structured questions).
5. The system shall provide AI-powered sentiment analysis and executive summarisation of the complete testimonial corpus associated with any Space, returning a structured JSON report covering overall sentiment, a numeric score, key positives, key complaints, and a prose summary.
6. The system shall provide an AI-powered improvement coaching feature that identifies recurring complaint themes in the testimonial corpus and generates prioritised, actionable remediation recommendations.
7. The system shall generate contextually personalised AI reply drafts for individual testimonials and dispatch approved replies via email directly from the dashboard.
8. The system shall provide a Wall of Love display page and embed code generator supporting four layouts, five themes, and six font options with real-time preview.

-
9. The system shall implement a multi-layer spam prevention pipeline on the public testimonial submission endpoint.
 10. The system shall allow business owners to toggle a “liked” flag on individual testimonials to curate the Wall of Love selection.

3.1.2 Non-Functional Requirements

- **Performance:** The initial dashboard page load, measured from navigation to full interactive state, shall not exceed 2 seconds on a standard broadband connection with cold browser cache. AI feature completions shall return within 4 seconds under normal Groq API load conditions.
- **Scalability:** The RESTful API tier shall be stateless in design — carrying no session state in process memory — to enable horizontal scaling via process duplication behind a load balancer without architectural changes.
- **Security:** All dashboard management endpoints shall require a valid, server-signed JWT in the Authorization header. Passwords shall be stored as bcrypt hashes with a minimum cost factor of 10. SQL injection shall be prevented by exclusive use of Prisma’s parameterised query interface.
- **Usability:** The testimonial submission form shall be completable by a first-time, non-technical reviewer within three minutes without any written instructions or tutorial. All interactive elements shall have accessible labels and visible focus states.
- **Accessibility and Responsiveness:** The UI shall fully support both dark and light display themes and shall render correctly on viewport widths from 360px (mobile) to 1440px (desktop) without horizontal scrolling or content overflow.
- **Reliability:** The application shall handle external API failures (Groq, Cloudinary, Gmail SMTP) gracefully, returning informative error messages to the user rather than unhandled exceptions or blank screens.

3.2 Information Collection

The requirements specification above was informed by three complementary data collection methodologies, each contributing a distinct perspective:

- **Competitive platform analysis:** Hands-on evaluation sessions were conducted with Testimonial.to, Senja, and Boast.io over a period of two weeks. Each platform was exercised through its complete feature set: creating a test testimonial campaign, submitting test testimonials through the public form, exploring the management dashboard, and attempting to embed a display widget on a test webpage. This direct evaluation produced

the feature gap analysis summarised in Table 2.1 and identified specific interaction design patterns (and anti-patterns) that informed TestiQra’s UX decisions.

- **Informal structured interviews:** Conversations were conducted with three small business owners — a freelance graphic designer, the operator of an online tutoring service, and a local fitness coaching service provider — each of whom had attempted to collect customer testimonials in the past. Common themes that emerged across all three interviews included: (1) not knowing what questions to ask reviewers to elicit useful responses beyond generic praise; (2) having no convenient mechanism for understanding the overall pattern of sentiment from a batch of reviews without reading each one individually; and (3) the technical barrier of embedding a testimonial gallery on a self-hosted website. These pain points map directly to the three sub-problems identified in Section 2.1 and provided human validation for the prioritisation of AI content generation, sentiment analysis, and the Wall of Love embedding system as the platform’s defining features.
- **Academic literature review:** A systematic review of published research on sentiment analysis algorithms, prompt engineering techniques for structured LLM outputs, web application security (with particular focus on bot mitigation strategies), and REST API design patterns informed the technical architecture decisions described in Sections 3.3 through 3.8.

3.3 System Architecture

TestiQra is architected as a three-tier client-server system following the widely established separation-of-concerns principle that decouples presentation logic, business logic, and data persistence into independently deployable layers. This architectural decision provides several significant advantages: each tier can be scaled independently in response to differential load patterns; the API tier can serve multiple front-end clients (web, mobile, or third-party integrations) without modification; and the separation enforces clean interface contracts between tiers, making each layer independently testable and replaceable.

Client Tier (Presentation Layer): The front-end is a React.js single-page application (SPA) built with the Vite development toolchain. Vite provides significantly faster hot-module replacement during development compared to the legacy Create React App toolchain and generates optimised, tree-shaken production bundles. The SPA is served as a collection of static assets (HTML, JS, CSS) from a CDN-backed static host (Vercel in the reference deployment). All user interface state — including the current theme preference, authenticated user data, form inputs, and dashboard filters — is managed within React’s component state and context system. The client communicates exclusively with the Application Tier via REST API calls over HTTPS, carrying JWT tokens in the Authorization header for all protected requests. The Wall of Love display page is a separate, unauthenticated React route that can be rendered directly by an iframe embed without requiring any user session.

Application Tier (Business Logic Layer): The back-end is a Node.js server running the Express.js web framework. The choice of Node.js is architecturally motivated by its non-blocking, event-driven I/O model: because the API server spends the majority of its processing time *waiting* on I/O, database query responses, Groq API completions, Cloudinary upload acknowledgements, and Gmail SMTP handshakes, a single-threaded, non-blocking runtime can achieve very high concurrency without the memory overhead of a thread-per-request model. The Express application is structured into discrete route modules, each encapsulating the handlers for a specific domain: user authentication, Space management, testimonial collection, AI features, Wall of Love, and email dispatch. A shared authentication middleware module intercepts all protected routes and verifies JWT validity before passing the request to the domain handler.

The Application Tier acts as the sole integration point for four external services: the **Groq API** for LLM inference (called synchronously within the AI feature endpoints, with the response awaited before replying to the client); **Google OAuth 2.0** for identity federation (the server validates Google ID tokens server-side using the `google-auth-library`, never exposing the client secret to the browser); **Gmail SMTP** via the Nodemailer library for outgoing email dispatch; and **Cloudinary's REST API** for binary media upload, storage, and CDN delivery. All external service credentials are managed via environment variables and are never exposed in client-side code.

Data Tier (Persistence Layer): PostgreSQL serves as the primary relational database, accessed exclusively through Prisma ORM's type-safe query client. Prisma provides two key engineering advantages: its schema-first migration system maintains database schema evolution as version-controlled code (preventing schema drift between environments), and its generated TypeScript client eliminates an entire class of runtime type errors by providing compile-time query validation. All relational data — users, spaces, questions, and testimonials — is stored in PostgreSQL. Binary media (images and video files) is stored on Cloudinary, with only the Cloudinary-generated CDN URLs persisted in the database records.

This architectural configuration means that the system can be fully deployed on freely available cloud infrastructure: Vercel for the React SPA, Render for the Node.js API, Neon for serverless PostgreSQL, and Cloudinary's free tier for media storage.

3.4 Analysis and Development of the Actual Solution

3.4.1 Database Schema

The TestiQra database schema is designed around a Space-centric relational model that reflects the hierarchical ownership structure of the application's domain: a *User* owns one or more *Spaces*, each Space accumulates one or more *Testimonials*, and each Testimonial is associated with a set of metadata attributes that support both management and display operations.

The User entity is the root of the ownership hierarchy. Each user record stores the owner's

first name, unique email address, an optional hashed password (null for OAuth-authenticated users), an optional Google identity token (null for locally-authenticated users), an authentication provider discriminator field (`authProvider`), and automatic timestamp fields for record creation and last update. The optional nature of both password and `googleId` fields, enforced by the database schema's nullable constraints, allows the system to accommodate both authentication flows within a single user model without requiring separate tables.

The `Space` entity captures the complete configuration of a testimonial collection campaign. Its most architecturally significant attribute is `space_name`, which carries a unique constraint at the database level and doubles as the URL slug for the public submission form and Wall of Love display page. This means that the application can resolve Space-specific pages without a separate URL routing lookup: the slug in the URL is the primary key of the Space in all but UUID format. Additional fields capture branding assets (logo URL), collection content (header text, custom welcome message, structured questions stored in the `child question` table), and thank-you page configuration (image URL, title, and message shown to reviewers upon submission).

The `Testimonial` entity records each individual submission. Key fields include the submitter's name and email, a boolean discriminator (`isTextContent`) distinguishing text from video submissions, the main content string (testimonial text for text submissions; a description or caption for video submissions), image and user profile image URLs (Cloudinary CDN links), the numeric star rating (integer 1–5), a video URL (Cloudinary CDN link, empty string for text submissions), the parent `spaceId` foreign key, an automatic submission timestamp, and the `liked` boolean flag used to curate the Wall of Love selection.

The `question` entity stores the individual questions configured for each Space as separate child records rather than as an array field on the Space entity. This normalised design allows questions to be individually added, removed, and reordered without requiring a full Space update operation, and maintains referential integrity through a foreign key relationship back to the parent Space with cascade delete behaviour.

Foreign key constraints and cascade delete operations are defined throughout the schema to maintain referential integrity automatically: deleting a User cascades to delete all their Spaces; deleting a Space cascades to delete all its Questions and Testimonials; and deleting a Testimonial removes its associated record cleanly without orphaned data. This cascade behaviour is essential for maintaining database consistency when business owners exercise the Space deletion function from the dashboard.

3.5 Description of Various Modules

3.5.1 Module 1 — User Authentication

The authentication module is the security gateway of the entire application. It handles two distinct authentication flows: local (email/password) and Google OAuth 2.0, both of which terminate in the issuance of a server-signed JSON Web Token upon successful credential validation.

For local authentication, the registration handler accepts an email, first name, and password. The password is hashed using `bcrypt` before persistence. The sign-in handler retrieves the user record by email and compares the submitted password against the stored hash using `bcrypt`'s constant-time comparison. Upon a successful match, a JWT is signed using a server-side secret key, encoding the user's email and first name as claims, and returned to the client with a configurable expiration window.

For Google OAuth authentication, the client-side Google Sign-In button initiates the Google identity flow and receives a Google ID token upon user consent. This token is sent to the server's `/api/v1/user/google-signin` endpoint, where it is validated against Google's public key infrastructure using the `google-auth-library`. The validated payload yields the user's Google account ID, email, and name; the handler upserts a User record accordingly and issues a TestiQra JWT in exactly the same format as the local flow, ensuring that downstream middleware is authentication-flow-agnostic.

The shared `AuthMiddleware` module intercepts every request to a protected endpoint, extracts the Bearer token from the Authorization header, and calls `jwt.verify()` against the server's secret. A verified, unexpired token injects the user's email and name into the request object and passes control to the route handler. Expired or malformed tokens receive differentiated 401 error responses that allow the client to distinguish between a session expiration (prompting a silent re-authentication) and a security error (prompting a full sign-out).

3.5.2 Module 2 — Space Management

The Space management module provides the complete CRUD lifecycle for testimonial campaign configurations. The creation handler accepts a rich payload including the space name (validated for uniqueness at the database level with a Prisma `@unique` constraint), logo upload (if provided, the image is first uploaded to Cloudinary and only the resulting CDN URL is stored), header text, custom message, thank-you page configuration, and an array of question strings. Questions are created as individual `question` records in a Prisma nested create operation within the same database transaction as the Space creation, ensuring atomicity.

The update handler accepts the same payload structure and performs a Prisma upsert operation: existing questions are deleted and recreated to handle additions, removals, and reordering in a single operation. The deletion handler removes the Space record and relies on the schema's

cascade delete behaviour to propagate deletion to all dependent Questions and Testimonials automatically.

The Space fetch endpoint returns all Spaces owned by the authenticated user, including their associated question arrays, in a single JOIN query. A separate Space info endpoint resolves a Space by its name slug and returns the full configuration for display to a reviewer on the public submission form.

3.5.3 Module 3 — Testimonial Collection

The testimonial collection module serves the public-facing submission endpoint and is the sole unauthenticated write surface in the application. Its handler is responsible for executing the spam prevention pipeline (described in detail in Section 4.4.3), processing any file uploads to Cloudinary, and persisting the testimonial record to the database.

File upload handling is performed using the Multer middleware, which buffers uploaded files in memory before the handler passes them to Cloudinary’s Node.js SDK for upload. The Cloudinary SDK returns a structured response including the permanent CDN URL of the uploaded asset; only this URL is stored in the Testimonial record, keeping the database free of binary data. The module handles three distinct upload scenarios: text testimonial with optional image attachment, text testimonial with optional profile photo, and video testimonial with optional profile photo.

The testimonial fetch endpoint, which is JWT-protected and returns all testimonials for a given Space ordered by submission timestamp, supports the management dashboard’s display requirements. The liked-toggle endpoint accepts a testimonial ID and flips the boolean liked field, updating the Wall of Love selection in real time.

3.5.4 Module 4 — AI Features

The AI features module is a single Express router mounted at `/api/v1/ai` that acts as a structured proxy between the TestiQra front-end and the Groq Llama 3.3-70B inference API. The module is intentionally stateless and prompt-driven: the front-end composes the complete prompt string — incorporating Space configuration data, testimonial corpus content, or individual testimonial details depending on the feature being invoked — and sends it as the body of a POST request. The router appends a fixed system instruction mandating JSON-only output and forwards the composed message to the Groq API with a constrained temperature setting (0.4) for output consistency and the `json_object` response format flag for structural reliability.

Five distinct prompt templates are maintained in the front-end codebase and dispatched to this single endpoint depending on the active AI feature: Space content generation, sentiment summary, improvement coaching, testimonial curation recommendation (which analyses the corpus and suggests which testimonials to include in the Wall of Love), and reply drafting. The response cleaning step strips any residual markdown code fence characters that may appear despite the JSON-mode instruction, and the cleaned JSON string is returned to the front-end for

parsing and display.

3.5.5 Module 5 — Wall of Love

The Wall of Love module renders a standalone, unauthenticated React page at the route `/testimonialwall/:spacename`. This page fetches only the liked testimonials from the public-accessible (but Space-name-keyed) Wall of Love API endpoint, formats them according to the selected display configuration, and renders the gallery without any authentication context, making it directly embeddable via `iframe`.

The display configuration — layout mode, theme, font, and size parameters — is passed as URL query parameters, allowing the embed snippet to encode the business owner's preferred configuration within the `iframe src` attribute. Four layout modes are implemented: *Marquee* (a horizontally scrolling infinite ticker), *Spotlight* (a large, prominent single-testimonial focus view with navigation), *Cascade* (a masonry-style multi-column layout), and *Grid* (a uniform rectangular grid). Five themes control the colour palette: Light, Dark, System (following the viewer's OS preference), Neon, and Minimal. Six font families allow alignment with common brand typography choices.

The embed code generator produces both an `iframe` snippet and a JavaScript snippet that dynamically inserts the `iframe`, accommodating website platforms (such as WordPress) that sanitise `iframe` tags in page editors.

3.5.6 Module 6 — Email Dispatch

The email dispatch module enables business owners to send AI-drafted reply emails to reviewers directly from the TestiQra dashboard, without switching to an external email client. The handler receives the recipient email address, subject line, and body text from the authenticated dashboard client, validates the request payload using Zod schema validation, and passes the message to a Nodemailer transporter configured with the business owner's Gmail account credentials (provided as environment variables).

Gmail SMTP is used with App Password authentication rather than the owner's primary Google account password, following Google's recommended practice for third-party SMTP access. The transporter configuration specifies TLS encryption for the SMTP connection. Successful dispatch returns a 200 response with a delivery confirmation; SMTP errors are caught and returned as structured 500 responses with informative error messages.

3.6 System Development Tools

The development environment for TestiQra comprised the following tools, each selected for a specific technical or productivity justification:

- **IDE — Visual Studio Code:** VS Code's extension ecosystem provides first-class support

for all technologies in the stack: the official Prisma extension provides schema syntax highlighting and auto-complete; the ESLint extension enforces code style rules at the editor level; the Tailwind CSS IntelliSense extension provides utility class auto-completion; and the Prettier extension formats code on save. The integrated terminal eliminates context switching during development.

- **Version Control — Git with feature-branch workflow:** All development work is tracked in a Git repository following a structured feature-branch workflow: new features are developed on named branches (e.g., `feature/ai-sentiment-summary`) and merged to the main branch only after self-review. This practice maintains a clean commit history and provides rollback capability if a feature implementation introduces regressions.
- **Package Manager — npm v10:** npm manages dependencies for both the front-end and back-end workspaces. The `package-lock.json` file is committed to the repository to ensure reproducible dependency resolution across environments.
- **Database GUI — Prisma Studio:** During development, Prisma Studio provides a browser-based graphical interface for inspecting and manually modifying database records. This accelerates debugging of data-layer issues by eliminating the need to write raw SQL queries for inspection purposes.
- **API Testing — Postman:** All REST API endpoints were systematically tested using Postman collections that mirror the full request structure of the front-end client. Postman's environment variable system was used to manage JWT tokens and base URL configuration across development and staging environments.

3.7 Choice of Language

- **JavaScript (ES2022+ / ECMAScript Modules):** The decision to use JavaScript for both the front-end and back-end is a deliberate architectural choice rather than a default. A single language across the full stack eliminates the cognitive overhead of context-switching between type systems, syntax conventions, and idioms during development. It also enables the sharing of validation schemas (Zod definitions) between the front-end form validation layer and the back-end request validation layer without code duplication. Node.js's npm ecosystem provides the largest package registry in software development, offering a mature, well-maintained package for virtually every integration requirement in the stack.
- **SQL via Prisma ORM:** PostgreSQL is accessed exclusively through Prisma's generated, type-safe query client. The decision to avoid raw SQL entirely is motivated by security (Prisma parameterises all queries by default, preventing SQL injection) and maintainability (Prisma's schema-first migration system ensures that the application code and database schema are always in sync). Prisma's query results are fully typed, enabling IDE-level auto-completion and compile-time error detection for data access code.

-
- **CSS via Tailwind CSS with Design Tokens:** Tailwind’s utility-first CSS framework eliminates the need to write custom CSS class names for the majority of styling tasks, dramatically accelerating UI development. CSS custom properties (design tokens) are used to implement the dual-theme system: token values are swapped at the root level when the theme is toggled, and all component colours reference tokens rather than hard-coded values, ensuring seamless dark/light switching without conditional class logic.

3.8 Choice of System for Implementation

- **React.js (Vite):** React’s component-based architecture is well-suited to the dashboard’s information-dense, highly interactive interface requirements. The ability to compose complex views from reusable, independently testable components reduces duplication and simplifies the maintenance of consistent interaction patterns across the application. Vite’s build pipeline produces significantly smaller production bundles and faster cold-start development servers compared to webpack-based alternatives.
- **Node.js / Express.js:** The non-blocking I/O model of Node.js is particularly advantageous for the TestiQra API server’s workload profile, which is I/O-bound rather than CPU-bound. The server spends the majority of its active time waiting for responses from PostgreSQL, Groq, Cloudinary, and Gmail — all of which are network-dependent operations. Node’s event loop can service dozens of concurrent such operations without the thread-per-request overhead of traditional synchronous server models.
- **PostgreSQL:** The relational structure of the testimonial data model — with its ownership hierarchies, one-to-many relationships, and integrity constraints — maps naturally to a relational database’s foreign key and constraint system. PostgreSQL was chosen over MongoDB specifically because the strict schema enforcement and transaction support of a relational database provide stronger guarantees for data integrity in a multi-tenant application where one user’s data must never be visible to another.
- **Groq API (Llama 3.3-70B):** The Groq inference platform was selected over OpenAI’s API primarily on the basis of inference latency. Groq’s custom LPU (Language Processing Unit) hardware achieves token generation speeds significantly exceeding GPU-based inference services, yielding average completion times for the Space generation prompt of under 2 seconds — well within the 4-second non-functional requirement. Groq’s free tier provides generous rate limits sufficient for academic development and small-scale production use without requiring a billing arrangement.

Table 3.1 provides a complete summary of the technology stack.

Table 3.1: Complete Technology Stack

Layer	Technology	Version
Frontend Framework	React.js (Vite)	v18.3
Routing	React Router DOM	v6
HTTP Client	Axios	v1.x
Styling	Tailwind CSS	v3
Drag-and-Drop	react-beautiful-dnd	v13
Toast Notifications	react-hot-toast	v2
Backend Framework	Express.js	v4.21
Runtime	Node.js (ESM)	v18+
ORM	Prisma	v5.18
Database	PostgreSQL	v15
AI / LLM	Groq SDK (Llama 3.3)	v1.1.2
Media Storage	Cloudinary	REST API
Authentication	JSON Web Token	v9
OAuth	google-auth-library	v9
Email	Nodemailer	v8
Validation	Zod	v3.23

3.9 REST API Specification

Table 3.2 documents all primary REST API endpoints exposed by the TestiQra Express.js server. All endpoints returning data sensitive to the authenticated user require a valid `Authorization: Bearer <JWT>` header; public endpoints (used by the reviewer submission form and the Wall of Love display page) require no authentication.

Table 3.2: REST API Endpoint Reference

Method	Endpoint	Auth	Description
POST	/api/v1/user/signup	No	Register new user
POST	/api/v1/user/signin	No	Authenticate, receive JWT
POST	/api/v1/user/google-signin	No	Google OAuth sign-in
POST	/api/v1/space-creation	JWT	Create a new Space
GET	/api/v1/space-fetch	JWT	List all Spaces for user
GET	/api/v1/spaceinfo	JWT	Fetch Space and questions
GET	/api/v1/spaceinfo/edit	JWT	Fetch Space for editing
PUT	/api/v1/edit	JWT	Update existing Space
DELETE	/api/v1/space/:name	JWT	Delete Space and all data
POST	/api/v1/sendtestimonials	No	Submit a testimonial (public)
GET	/api/v1/fetchtestimonials	JWT	Retrieve all testimonials
POST	/api/v1/liked	JWT	Toggle liked flag
POST	/api/v1/ai	No	Groq LLM completion
POST	/api/v1/email/send-reply	JWT	Dispatch AI reply email

Chapter 4

System Implementation

4.1 Hardware Requirements

The following hardware specifications define the minimum environment required to develop, build, and run the TestiQra application in a local development context. Production deployment requirements are substantially lower, as the application is designed to run on cloud-hosted virtual machines with dedicated resources.

Table 4.1: Minimum Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 (8th Gen) or equivalent, 1.6 GHz
RAM	4 GB (8 GB recommended for development)
Storage	10 GB free disk space (SSD preferred)
Network	Broadband internet connection (required for AI and cloud APIs)
Display	1366 × 768 resolution or higher

An SSD is strongly recommended over an HDD for the development environment, as Node.js module installation (which creates tens of thousands of small files in the `node_modules` directory) and Prisma schema compilation are both heavily dependent on file system throughput. The broadband network requirement is non-negotiable in development because all five AI features, media uploads, and OAuth sign-in require live internet access to external APIs.

4.2 Software Requirements

Table 4.2: Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Runtime	Node.js v18.0 or higher
Database	PostgreSQL v14.0 or higher
Package Manager	npm v9.0 or higher
Browser	Chrome 110+, Firefox 110+, or Edge 110+
IDE	Visual Studio Code (recommended)

Node.js v18 or higher is required specifically because the back-end codebase uses native ECMAScript Module (ESM) syntax (`import/export`) rather than CommonJS (`require/module.exports`), and stable ESM support in Node.js was introduced in v12 but reached production maturity in v18. The browser version requirements are driven by the use of the MediaRecorder Web API for in-browser video recording, which requires a modern browser implementation. Internet Explorer is not supported.

4.3 Input Requirements

The TestiQra system accepts structured inputs from three distinct user categories, each with different data types and validation requirements:

- **Business owner inputs (authenticated):** During account creation, the system accepts the owner's first name, email address, and a password of minimum 8 characters (or delegates identity to Google OAuth). During Space creation and editing, the system accepts a globally unique Space name (slug), an optional logo image file (validated for MIME type and file size before Cloudinary upload), a header string of up to 100 characters, a custom welcome message of up to 500 characters, thank-you page configuration fields, and an array of up to ten question strings. All text inputs are validated server-side using Zod schemas that enforce length limits, character set restrictions, and required/optional classifications.
- **Reviewer inputs (unauthenticated):** The public submission form accepts the reviewer's full name and email address, an integer star rating between 1 and 5, a testimonial text body (minimum 20 characters to discourage single-word submissions), an optional testimonial image file, an optional profile photo, and optionally a video file (or a browser-recorded video blob) in place of the text testimonial. The form also includes a required consent checkbox confirming the reviewer's consent to the testimonial being published. A honeypot hidden field is embedded in the form to detect automated bots.
- **AI feature inputs:** The Space content generation feature accepts four company metadata fields: company name, business type, target audience description, and a brief product/service description. The sentiment summary and improvement coaching features accept the serialised testimonial corpus (all testimonial texts for the Space, formatted as a numbered list). The reply drafting feature accepts the individual reviewer's name, star rating, and testimonial text. These inputs are incorporated into template prompt strings on the front-end before dispatch to the AI router.

4.4 Key Code Implementations

4.4.1 JWT Authentication Middleware

The JWT authentication middleware is implemented as a shared Express.js middleware function that is applied to every protected route in the application. The middleware extracts the Bearer token from the incoming request's `Authorization` header and calls the `jsonwebtoken` library's `verify()` method against the server's secret key.

The implementation handles three distinct error cases independently: a missing or malformed `Authorization` header (returning 401 with "Authorization required"), a token whose claims do not include the expected `email` and `firstname` fields (returning 401 with "Invalid token"), and an expired token (returning 401 with "Token expired", differentiated from a forgery error to allow the client to silently refresh the session). On successful verification, the middleware injects the decoded `email` and `firstname` claims into the `req` object, making them available to all downstream route handlers without requiring repeated token parsing.

This design ensures that the authenticated user's identity is always available within any protected handler without any additional database lookup, as the JWT itself carries the identity claims. This is the principal performance advantage of stateless JWT authentication over server-side session management: no database round-trip is required to identify the authenticated user on each request.

4.4.2 Groq AI Integration

The Groq AI router implements a prompt-driven, JSON-mode LLM integration that is simultaneously general enough to serve all five AI features through a single endpoint and disciplined enough to guarantee parseable outputs regardless of the semantic content of the prompt.

The key implementation decisions are: (1) setting the `temperature` parameter to 0.4 rather than the default 1.0, which reduces output variability and makes the model's responses more consistent and less prone to structural deviations from the requested JSON schema; (2) setting `response_format` to `json_object`, which activates the model's constrained decoding mode and prevents it from generating any non-JSON output; (3) using a fixed system instruction that reinforces the JSON-only constraint at the system prompt level as an additional safeguard; and (4) post-processing the response to strip any residual markdown code fence markers that may appear despite the constrained decoding, ensuring that the output can be passed directly to `JSON.parse()` without error.

The `max_tokens` parameter is set to 1024, which provides sufficient token budget for the longest expected response (the improvement coaching report, which may contain five to eight detailed issue objects) while preventing runaway generation that could exceed acceptable response latency thresholds.

4.4.3 Multi-Layer Spam Prevention Pipeline

The spam prevention pipeline is one of the most technically distinctive aspects of TestiQra, implementing four independent defence mechanisms that operate sequentially on every incoming testimonial submission request. The pipeline is designed so that each layer targets a different class of bot behaviour, maximising rejection coverage while minimising false positives that would penalise genuine human submitters.

Layer 1 — Honeypot Field Detection: The submission form includes two hidden input fields (`_gotcha` and `website`) that are invisible to human users via CSS but are typically populated by automated form-filling bots that process all input fields in the HTML DOM. Any request that arrives with non-empty values in these fields is silently accepted with a success response (HTTP 200) rather than rejected with an error code. This “silent accept” strategy prevents bots from adapting to the filter by detecting a rejection response; from the bot’s perspective, the submission succeeded, discouraging retry attempts.

Layer 2 — IP-Based Rate Limiting: Every incoming request’s source IP address is extracted from the `x-forwarded-for` header (set by proxy or load balancer) or directly from the socket. A timestamp-keyed submission history is maintained in a Map data structure keyed by IP address. If the number of submissions from a given IP within the preceding one-hour window exceeds 10, the request is rejected with HTTP 429. Furthermore, if a single IP accumulates 30 or more total recorded submissions (across any time window), it is added to a permanent blocklist Set for the lifetime of the server process, and all subsequent requests from that IP are rejected without inspecting the submission content.

Layer 3 — Email-Based Rate Limiting: Independently of the IP check, the submitted email address is normalised (lowercased and whitespace-trimmed) and checked against a parallel submission history Map keyed by email. A maximum of 3 submissions per unique email address per 24-hour window is enforced. This layer catches bots that cycle through different IP addresses (e.g., via rotating proxy networks) but reuse the same email address, and also prevents human users from gaming the rating system through repeated submissions.

Layer 4 — Content Pattern Filtering: The testimonial text body is tested against a set of regular expression patterns associated with high-confidence spam indicators: the presence of any URL (matching `https?://`), the occurrence of known spam-associated keywords (including pharmaceutical names, gambling terms, and cryptocurrency solicitation vocabulary), and character repetition sequences longer than six characters (a common bot-generated content marker). Testimonials matching any pattern are rejected with HTTP 400. This layer catches bots that pass the rate limiting layers but generate textually invalid content.

The cumulative effect of all four layers, as measured in the empirical evaluation described in Chapter 5, is a rejection rate exceeding 95% of simulated bot submissions while maintaining a zero false-positive rate on a representative set of legitimate human submission samples.

4.5 Operation Manual

4.5.1 For Business Owners (Dashboard Users)

The following procedure describes the complete workflow for a business owner from initial account creation to embedded gallery publication:

1. **Account creation:** Navigate to the TestiQra application URL and click “Create Account.” Enter a first name, email address, and password, or click “Continue with Google” to use an existing Google account. Upon successful registration, the application redirects to the main dashboard.
2. **Space creation — AI-assisted:** Click the “Create Space” button in the dashboard header. Select the “Create with AI” option. In the AI configuration panel, enter the company name, select the business type from the dropdown, describe the target audience (e.g., “small business owners aged 25-45”), and provide a brief description of the product or service. Click “Generate.” Within approximately 2 seconds, the form fields are populated with an AI-generated header, welcome message, and five targeted questions — all of which are fully editable before submission.
3. **Space creation — manual:** Alternatively, select “Create from Scratch” and fill in all Space configuration fields manually, including the Space slug name, logo upload, header, welcome message, thank-you page content, and questions.
4. **Testimonial collection:** From the Space detail page, copy the generated submission URL (formatted as `/testimonial.in/:spacename`) and share it with customers via email, social media, or QR code.
5. **Dashboard management:** Return to the Space’s testimonial management view to see all incoming submissions displayed in chronological order. Use the filter and search controls to locate specific testimonials by keyword, rating, or submission date.
6. **AI analysis:** Click the “AI Summary” button to request a sentiment analysis report for the entire testimonial corpus. The report displays the overall sentiment label, a 0–100 sentiment score, key positive themes, key complaint themes, and an executive summary paragraph. Click “Improvement Coach” for a prioritised remediation action plan.
7. **Wall of Love curation:** Click the heart icon on individual testimonials to mark them as “liked” and add them to the Wall of Love gallery.
8. **Gallery embedding:** Navigate to the Space’s Wall of Love page and use the real-time customiser panel to select a theme, layout, font, and size parameters. When satisfied with the preview, click “Generate Embed Code” to produce both an iframe snippet and a JavaScript snippet. Paste the chosen snippet into the business’s website editor.
9. **AI reply dispatch:** For individual testimonials, click the “Reply” button to generate an

AI-drafted personalised response. Review the draft, edit if desired, and click “Send” to dispatch the reply directly to the reviewer’s email address.

4.5.2 For Reviewers (Submission Form Users)

1. Open the Space URL shared by the business (accessible from any device with a modern web browser; no account or registration required).
2. Select a star rating from 1 (poor) to 5 (excellent) by clicking the corresponding star icon.
3. Either type a testimonial in the text area, or switch to video mode by toggling the “Video” tab and either recording a video directly in the browser using the built-in recorder or uploading a pre-recorded video file.
4. Optionally, click “Add Photo” to attach an image (such as a product photo or project screenshot) to the testimonial.
5. Optionally, upload a profile photo that will be displayed alongside the testimonial in the Wall of Love gallery.
6. Enter your full name and email address in the respective fields.
7. Read and check the consent checkbox confirming permission for the testimonial to be published.
8. Click “Submit.” Upon successful submission, the thank-you page configured by the business is displayed.

4.6 Output Reports and Artefacts

The TestiQra system produces the following distinct output artefacts as a result of its core operations:

- **AI Sentiment Summary Report:** A structured JSON document containing five fields: `overallSentiment` (one of Positive, Mixed, or Negative), `sentimentScore` (an integer from 0 to 100, where 0 is maximally negative and 100 is maximally positive), `keyPositives` (an array of strings identifying recurring positive themes), `keyComplaints` (an array of strings identifying recurring negative themes), and `summary` (a 2–3 sentence prose executive summary suitable for sharing with stakeholders). This report is rendered in the dashboard as a structured card with visual indicators.
- **AI Improvement Coach Report:** A structured JSON array in which each element represents a distinct recurring complaint theme identified in the testimonial corpus. Each element includes the theme name, a priority level (High, Medium, or Low), a detailed description of the issue as manifested in the testimonials, and a concrete, actionable

remediation recommendation. This report is presented as a prioritised checklist in the dashboard.

- **AI Reply Draft:** A 2–3 sentence personalised email reply generated for a specific reviewer, incorporating their name, the star rating they provided, and contextual references to the content of their testimonial. The draft is presented in an editable text area within the dashboard, allowing the business owner to review and refine before dispatching.
- **Wall of Love Embed Code:** Two alternative embed snippets generated for each customised gallery configuration: (a) an `<iframe>` tag with the gallery URL and configuration parameters encoded as query string values, suitable for direct insertion into HTML editors; and (b) a JavaScript snippet that programmatically creates and inserts the iframe, suitable for website builders that sanitise HTML iframe tags.
- **Testimonial Dashboard View:** The complete testimonial corpus for any Space, presented in the management dashboard as a scrollable, searchable, filterable list with each testimonial displayed as a card showing the reviewer’s name, profile photo (if provided), star rating, submission timestamp, content preview, and like/unlike and reply action buttons.

Chapter 5

Testing

5.1 API Route Testing Methodology

All backend routes were tested systematically using Postman. Each endpoint was exercised with a combination of valid payloads, invalid inputs, missing fields, and unauthorised requests to verify both functional correctness and error handling. For each route, at minimum the following three scenarios were tested: a successful request with a well-formed payload; a request with missing required fields; and a request carrying an invalid or absent JSON Web Token (JWT). This approach ensures that both the *happy-path* behaviour and the defensive error-handling logic of the middleware layer were verified independently.

Response time measurements were taken using Postman’s built-in response time monitor across ten (10) consecutive requests per endpoint under idle server load, with no concurrent users present. The values reported represent the arithmetic mean of those ten measurements. For AI feature endpoints, measurements were taken at different times of day to account for variance in Groq API latency.

The following route groups were tested in full.

5.1.1 User Authentication Routes

The routes residing under the `/api/v1/user` prefix — specifically `POST /signup`, `POST /signin`, and `POST /google-signin` — were tested with valid credentials, duplicate email addresses, malformed email formats, passwords below the minimum length of six characters, and entirely absent request body fields. JWT token generation and the response payload structure were verified for all successful authentication flows.

5.1.2 Space Management Routes

`POST /api/v1/space-creation` and `PUT /api/v1/edit` were tested with complete payloads, partial payloads omitting required fields such as header and spacename, and with an expired or missing Bearer token to confirm correct rejection by the authentication middleware. `DELETE /api/v1/space/:space_name` was tested to verify cascading deletion of all associated question and testimonial records, ensuring referential integrity is maintained in the PostgreSQL schema.

5.1.3 Space Fetching Routes

GET `/api/v1/space-fetch` was tested to confirm that the endpoint returns only the spaces belonging to the currently authenticated user, enforcing per-user data isolation. GET `/api/v1/spaceinfo` and GET `/api/v1/spaceinfo/edit` were tested with both valid space name query parameters and non-existent space names to verify graceful null-response handling.

5.1.4 Testimonial Submission and Retrieval Routes

POST `/api/v1/sendtestimonials` was tested with the following distinct payload categories:

1. A valid text-only testimonial submission.
2. A valid video-only testimonial submission (with a Cloudinary URL).
3. A combined text and video testimonial.
4. A submission with the honeypot field (`_gotcha`) populated, simulating unsophisticated automated bot behaviour.
5. A submission containing a URL (`https://`) within the testimonial body, to verify Layer 4 content filtering.
6. A submission triggering the per-IP rate limit (more than ten requests per hour from the same address).
7. A submission triggering the per-email rate limit (more than three submissions per 24-hour window from the same address).

GET `/api/v1/fetchtestimonials` was verified against both existing and non-existent space names. POST `/api/v1/liked` was tested with valid testimonial identifiers as well as non-existent identifiers to confirm correct database update behaviour.

5.1.5 AI Inference Route

POST `/api/v1/ai` was tested with a valid, well-structured prompt; an empty prompt body to verify the 400-level validation response; and a simulated missing Groq API key (achieved by temporarily unsetting the `GROQ_API_KEY` environment variable) to verify that the server returns a meaningful 500-level error rather than crashing.

5.1.6 Email Reply Route

POST `/api/v1/email/send-reply` was tested with a complete payload containing the `to`, `subject`, and `body` fields, and with requests omitting each required field individually to confirm that 400-level validation errors are raised correctly.

Chapter 6

Results and Discussion

6.1 Brief Evaluation of the System

The overall evaluation of TestiQra demonstrates that the platform successfully fulfils its primary objectives of simplifying testimonial collection, improving user experience, and reducing manual effort through AI-assisted automation. The integration of AI-generated testimonial forms significantly reduces the time required for Space setup while maintaining content quality and contextual relevance for different business domains.

The system performs effectively in handling multimedia testimonials through Cloudinary integration, enabling users to collect both text-based and video-based feedback within a centralised environment. The responsive dashboard layout and streamlined workflow contribute positively to usability for both administrators and end users.

From a security and reliability perspective, the multi-layer spam prevention pipeline provides strong protection against automated spam submissions and malicious content. The combined use of honeypot validation, IP-based rate limiting, email-based rate limiting, and content filtering helps preserve the authenticity and overall quality of collected testimonials.

In terms of technical performance, the platform delivers satisfactory response times for core operations such as dashboard loading, Space retrieval, AI content generation, and testimonial summarisation. The adoption of modern technologies including React, Node.js, PostgreSQL, Prisma, Redis, and Groq AI contributes to scalable and efficient system behaviour.

Overall, the evaluation indicates that TestiQra is a practical, user-friendly, and technically capable testimonial management platform suitable for modern digital applications requiring automated feedback collection and AI-assisted content generation.

6.2 Interpretation of Results

6.2.1 Performance Benchmarks

Table 6.1 presents the complete API performance benchmark results obtained through the Postman methodology described in Section 5.1.

Table 6.1: System Performance Benchmarks

Metric	Value	Conditions / Testing Source
Initial page load (dashboard)	1.4 s	Chrome DevTools Lighthouse (cold cache)
API response – Space fetch	87 ms	Postman, 10-request mean, 10 Spaces
AI completion – Space generation	1.8 s	Postman, 10-request mean, Groq Llama 3.3
AI completion – Summary (30 items)	3.1 s	Postman, 10-request mean, Groq Llama 3.3
Video upload (50 MB)	8.2 s	Postman upload timer, Cloudinary free tier
Spam rejection rate	95.4%	Postman Collection Runner, 1 000 requests

6.2.2 Route-Level Functional Test Results

Table 6.2 presents the pass/fail summary for all 25 tested route scenarios. A scenario is marked as *Pass* if the actual HTTP status code and response body structure matched the expected values in every one of the ten consecutive Postman runs.

Table 6.2: Route-Level Functional Test Results

Route	Scenario	Expected Status	Result
POST /user/signup	Valid new user	201	Pass
POST /user/signup	Duplicate email	400	Pass
POST /user/signup	Missing password	400	Pass
POST /user/signin	Valid credentials	200	Pass
POST /user/signin	Incorrect password	401	Pass
POST /space-creation	Valid payload + token	200	Pass
POST /space-creation	Missing header field	400	Pass
POST /space-creation	No auth token	401	Pass
PUT /edit	Valid update payload	200	Pass
PUT /edit	Non-existent space	404	Pass
DELETE /space/:name	Valid space name	200	Pass
GET /space-fetch	Authenticated user	200	Pass
GET /spaceinfo	Valid space name	200	Pass
GET /spaceinfo	Non-existent space	Null handled	Pass
POST /sendtestimonials	Valid text submission	201	Pass
POST /sendtestimonials	Valid video submission	201	Pass

Table 6.2 (continued)

Route	Scenario	Expected Status	Result
POST /sendtestimonials	Honeypot field populated	200 (silent)	Pass
POST /sendtestimonials	URL in testimonial body	400	Pass
POST /sendtestimonials	IP rate limit exceeded	429	Pass
POST /sendtestimonials	Email rate limit exceeded	429	Pass
GET /fetchtestimonials	Valid space name	201	Pass
POST /liked	Valid testimonial ID	201	Pass
POST /ai	Valid structured prompt	200	Pass
POST /ai	Empty prompt body	400	Pass
POST /email/send-reply	Valid payload	200	Pass
POST /email/send-reply	Missing to field	400	Pass

All 25 tested scenarios produced the expected HTTP status codes and response body structures. No route returned an unhandled 500-level error under normal operating conditions. Notably, the spam rejection pipeline tested via Postman’s Collection Runner dispatching 1,000 requests across five bot behaviour profiles achieved an overall rejection rate of 95.4%, with a 0% false-positive rate recorded against 50 legitimate control submissions. This confirms that the IP-based rate limiting, per-email rate limiting, honeypot detection, and content-filter thresholds are calibrated conservatively enough to avoid penalising genuine users under realistic usage patterns, whilst rejecting the overwhelming majority of automated submissions.

6.3 Application Areas and Scope of the Work

6.3.1 SaaS and Software Products

Software companies represent the primary target demographic for TestiQra, as they typically maintain established customer email lists from which testimonial requests can be dispatched, and as social proof has been documented as a particularly high-value conversion driver in the SaaS sales funnel where purchase decisions involve ongoing subscription commitments rather than one-time transactions.

For SaaS operators, TestiQra’s AI summary feature serves a dual purpose: as a marketing

intelligence tool, it surfaces the specific value propositions most frequently cited by satisfied customers (informing landing page copy and advertising messaging); as a product management tool, it surfaces the most frequently recurring pain points (informing engineering backlog prioritisation). The Improvement Coach feature provides a direct translation from testimonial sentiment to actionable engineering and customer success recommendations. The Wall of Love embed can be deployed on product landing pages, pricing pages, and G2/Capterra listing descriptions to maximise social proof exposure at high-intent decision points in the customer journey.

6.3.2 E-Commerce

Online retailers can deploy product-specific Spaces, one per SKU or product category, to collect post-purchase testimonials from verified buyers. The video testimonial capability is particularly valuable in e-commerce contexts, where visual authenticity (a customer showing the product in real use) carries substantially higher trust weight than text reviews, which are more susceptible to perceived inauthenticity. The AI Improvement Coach is positioned to surface fulfilment issues (slow shipping, damaged packaging) and customer service pain points from testimonial text, providing actionable operational insights that standard star rating aggregations cannot capture.

6.3.3 Coaching and Consulting

Individual service professionals, executive coaches, business consultants, life coaches, and independent trainers, benefit disproportionately from high-quality testimonials because their services are inherently intangible and trust-dependent. TestiQra's branded Space pages allow these professionals to create a polished, professional testimonial collection experience that reflects their brand positioning. The video testimonial format is particularly compelling for this segment, as a 30-second video of a satisfied client speaking authentically carries more persuasive weight than any text testimonial for service businesses where personal chemistry and results are the primary purchase criteria. The AI reply drafting feature enables prompt, personalised acknowledgement of every reviewer, reinforcing the relationship with existing clients and increasing the likelihood of referrals.

6.3.4 Educational Institutions

Universities, business schools, and online course providers can deploy Spaces for systematic collection of programme and course feedback from students. Compared to traditional course evaluation surveys (which are collected anonymously and aggregated statistically), testimonial collection provides richer qualitative data that can be used in accreditation documentation, programme marketing materials, and prospective student recruitment. The AI sentiment summary provides semester-level aggregate insight in seconds, supporting quality assurance and programme improvement workflows without the manual synthesis effort currently required from programme directors.

6.3.5 Non-Profit Organisations

Non-profit organisations can use TestiQra to collect impact stories from programme beneficiaries, volunteers, and donors. The Wall of Love gallery, embedded on fundraising landing pages, provides powerful emotional social proof that is particularly effective in motivating charitable giving decisions. The open-source nature of the platform and its compatibility with free-tier infrastructure make it accessible to budget-constrained non-profits that cannot afford commercial testimonial platform subscriptions.

6.3.6 Recruitment and Employer Branding

Human resources departments and talent acquisition teams can deploy employer branding Spaces to collect employee testimonials for careers pages and LinkedIn profiles. The AI sentiment summary provides an aggregate view of perceived workplace culture themes, a valuable input for HR strategy and employer reputation management. Video employee testimonials for career site embedding are increasingly recognised as a high-impact recruitment marketing tool.

Chapter 7

Conclusion

7.1 Advantages and Special Features of the System

TestiQra represents a substantive advancement in the capability landscape of testimonial management platforms, delivering a combination of features that collectively distinguish it from every commercially available competitor identified in the literature survey:

- **End-to-end AI integration across all five workflow stages:** Rather than applying AI at a single point (such as sentiment classification of already-collected reviews), TestiQra deploys LLM intelligence at every stage of the testimonial lifecycle. The AI generates the collection campaign configuration, assists with corpus analysis during the management stage, coaches the business on operational improvements, curates the gallery for publication, and drafts personalised responses to individual reviewers. This holistic integration reduces manual cognitive effort at every workflow transition point, not just at individual stages.
- **Multi-modal testimonial collection in a unified form:** The submission form seamlessly supports both text testimonials (with optional image attachment) and video testimonials (browser-recorded or uploaded) within a single, polished public page. This flexibility increases the volume and diversity of testimonial submissions by accommodating different reviewer communication preferences.
- **Fully customisable, embeddable Wall of Love:** Five visual themes, four layout modes, six font families, and real-time parameter sliders — with live preview before embed code generation — give businesses complete control over the visual presentation of their gallery. This level of customisation significantly exceeds what any surveyed competitor offers.
- **Dual authentication without UX compromise:** Supporting both local password-based authentication and Google OAuth 2.0 from a single unified interface accommodates the full spectrum of user authentication preferences without requiring separate registration flows or account migration.
- **Robust four-layer spam prevention without CAPTCHA:** The honeypot plus dual rate-limiting plus content-filtering pipeline achieves a 95.4% rejection rate on simulated bot attacks with zero false positives on legitimate submissions — all without imposing any CAPTCHA friction on genuine reviewers.
- **Completely free-tier deployable and open-source:** The entire platform — front-end, API, database, media storage, and AI inference — can be deployed at zero cost using

free tiers of Vercel, Render, Neon, Cloudinary, and Groq respectively. The open-source codebase allows the research community and independent developers to audit, extend, and deploy their own instances.

7.2 Limitations

A rigorous academic assessment of TestiQra must acknowledge the following limitations, which represent engineering debt and design constraints that must be addressed before any production deployment serving real users:

- **In-memory rate limiting state:** The IP submission history and email submission history data structures are maintained in Node.js process memory rather than in a persistent external store such as Redis. This means that rate limit state is lost whenever the server process restarts (allowing a bot that was being rate-limited to resume attacks after a server restart), and that multi-instance deployments (required for horizontal scaling) would have inconsistent rate limit enforcement across instances.
- **Permissive CORS configuration:** The Express CORS middleware is currently configured to accept cross-origin requests from any domain. Production deployment must restrict the allowed origin list to the known front-end domain to prevent cross-origin API abuse.
- **Ownership verification gap on deletion:** The Space deletion endpoint currently verifies only that the requesting user is authenticated (via JWT) but does not confirm that the authenticated user is the actual owner of the Space being deleted. This represents a broken access control vulnerability that could allow one authenticated user to delete another user's Spaces.
- **No server-side pagination:** All testimonials for a given Space are returned in a single database query and HTTP response. For high-volume Spaces with hundreds or thousands of testimonials, this creates memory pressure on the server and increases response payload size to impractical levels.
- **AI feature dependency without fallback:** All five AI features are unavailable during periods of Groq API downtime. No local fallback mechanism or graceful degradation strategy is currently implemented; affected features simply display an error state in the dashboard.
- **Small-sample evaluation:** The empirical evaluation was conducted with five participants for the timing study and a programmatic simulator for spam testing. Both represent controlled, small-scale evaluations that may not fully capture the variability of real-world usage patterns. A larger-scale user study and extended production monitoring period would be required to substantiate the results with statistical confidence.

7.3 Future Extension

7.3.1 Short-Term Extensions (0–6 Months)

- **Redis-backed rate limiting:** Replace the in-process Map-based rate limit state with a Redis data store. Redis’s atomic increment operations and built-in key expiry (TTL) make it the industry-standard solution for distributed rate limiting. This change enables correct rate limit enforcement across multiple server instances and survives server restarts.
- **hCaptcha integration:** Add hCaptcha as an optional fifth spam prevention layer for Spaces owned by users who have experienced persistent bot activity that the existing four-layer pipeline is not fully suppressing. Keeping hCaptcha optional rather than mandatory preserves the frictionless experience for the majority of legitimate submitters.
- **Cursor-based pagination:** Implement cursor-based pagination on the testimonial fetch endpoint, returning testimonials in configurable page sizes (e.g., 20 per page) with a cursor token pointing to the next page. This eliminates the performance and memory scalability bottleneck identified in Section 6.2.
- **Ownership verification on all mutation endpoints:** Add a Space ownership check (verifying that `req.email` matches the `userId` foreign key of the Space being operated upon) to all Space and Testimonial mutation and deletion handlers.

7.3.2 Medium-Term Extensions (6–18 Months)

- **Analytics dashboard with time-series visualisation:** Build a dedicated analytics view per Space displaying testimonial submission volume over time, star rating distribution as a histogram, sentiment score trend as a time series, and video versus text submission ratio. These visualisations would give business owners longitudinal insight into how customer sentiment evolves in response to product changes or service improvements.
- **Multilingual support (i18n):** Extend the submission form interface and AI prompt templates to support multiple languages, enabling businesses operating in non-English markets to deploy TestiQra without requiring reviewers to interact in a second language.
- **CRM webhook integration:** Implement configurable outgoing webhook notifications that fire when a new testimonial is submitted, sending a structured payload to CRM systems (HubSpot, Salesforce, Zoho) to enable automated follow-up workflows within existing business tool stacks.
- **API key authentication:** Provide an API key authentication option (as an alternative to JWT) for programmatic access to the testimonial fetch and liked-toggle endpoints, enabling third-party integrations and automated workflows without requiring OAuth flows.

7.3.3 Long-Term Extensions (18+ Months)

- **React Native mobile applications:** Develop companion iOS and Android applications that give business owners push notification alerts on new testimonial submissions and allow testimonial management (liking, replying) from mobile devices.
- **Video testimonial AI analysis:** Integrate OpenAI Whisper (or an equivalent speech-to-text model) to automatically transcribe video testimonials, and apply multimodal LLM analysis to the transcript and video metadata for sentiment scoring and theme extraction. This would extend the AI analysis capabilities to the video testimonial corpus, which is currently excluded from sentiment analysis.
- **Stripe billing integration and tiered subscriptions:** Implement a freemium subscription model using Stripe for billing management. Free tier limits (e.g., three Spaces, 50 testimonials per Space) would encourage individual and small business adoption, while paid tiers would unlock unlimited Spaces, custom domain support for the submission form, priority AI processing, and advanced analytics.
- **White-label agency mode:** Develop an agency-tier account type that allows digital marketing agencies to rebrand the TestiQra platform with their own logo, domain, and colour scheme, and resell it as a managed service to their clients — a high-value product offering that requires no additional development effort from the agency.

Bibliography

- [1] BrightLocal. *Local Consumer Review Survey 2023*. BrightLocal, 2023. Available at: <https://www.brightlocal.com/research/local-consumer-review-survey/>
- [2] B. Pang and L. Lee. Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2):1–135, 2008.
- [3] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.
- [4] T. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.
- [5] P. Turney. Thumbs up or thumbs down? Semantic orientation applied to unsupervised classification of reviews. In *Proceedings of ACL 2002*, pages 417–424, 2002.
- [6] Meta Open Source. *React — The Library for Web and Native User Interfaces*, 2024. Available at: <https://react.dev>
- [7] Prisma Data Inc. *Prisma ORM Documentation*, 2024. Available at: <https://www.prisma.io/docs>
- [8] Groq, Inc. *Groq API Documentation*, 2024. Available at: <https://console.groq.com/docs>
- [9] Meta AI. *Introducing Meta Llama 3: The Most Capable Openly Available LLM to Date*. Meta AI Blog, 2024.
- [10] Cloudinary Ltd. *Cloudinary Developer Documentation*, 2024. Available at: <https://cloudinary.com/documentation>
- [11] Tailwind Labs Inc. *Tailwind CSS Documentation*, 2024. Available at: <https://tailwindcss.com/docs>
- [12] M. Jones, J. Bradley, and N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force, 2015.
- [13] D. Hardt. *RFC 6749: The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force, 2012.
- [14] Nodemailer. *Nodemailer Documentation*, 2024. Available at: <https://nodemailer.com/about/>
- [15] C. McDonnell. *Zod — TypeScript-first Schema Validation*, 2024. Available at: <https://zod.dev>

-
- [16] Express.js Contributors. *Express 4.x API Reference*, 2023. Available at: <https://expressjs.com/en/4x/api.html>
 - [17] Mozilla Developer Network. *MediaRecorder API*, 2023. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>
 - [18] The PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*, 2024. Available at: <https://www.postgresql.org/docs/15/>
 - [19] Senja. *Testimonial Collection Platform*, 2024. Available at: <https://senja.io>
 - [20] Testimonial.to. *Testimonial Collection Platform*, 2024. Available at: <https://testimonial.to>
 - [21] E. You et al. *Vite — Next Generation Frontend Tooling*, 2024. Available at: <https://vitejs.dev>
 - [22] Express.js Contributors. *Multer — Node.js Middleware for Handling Multipart/Form-Data*, 2024. Available at: <https://github.com/expressjs/multer>
 - [23] D. Provos and D. Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, 1999.

A1: Architectural Diagram

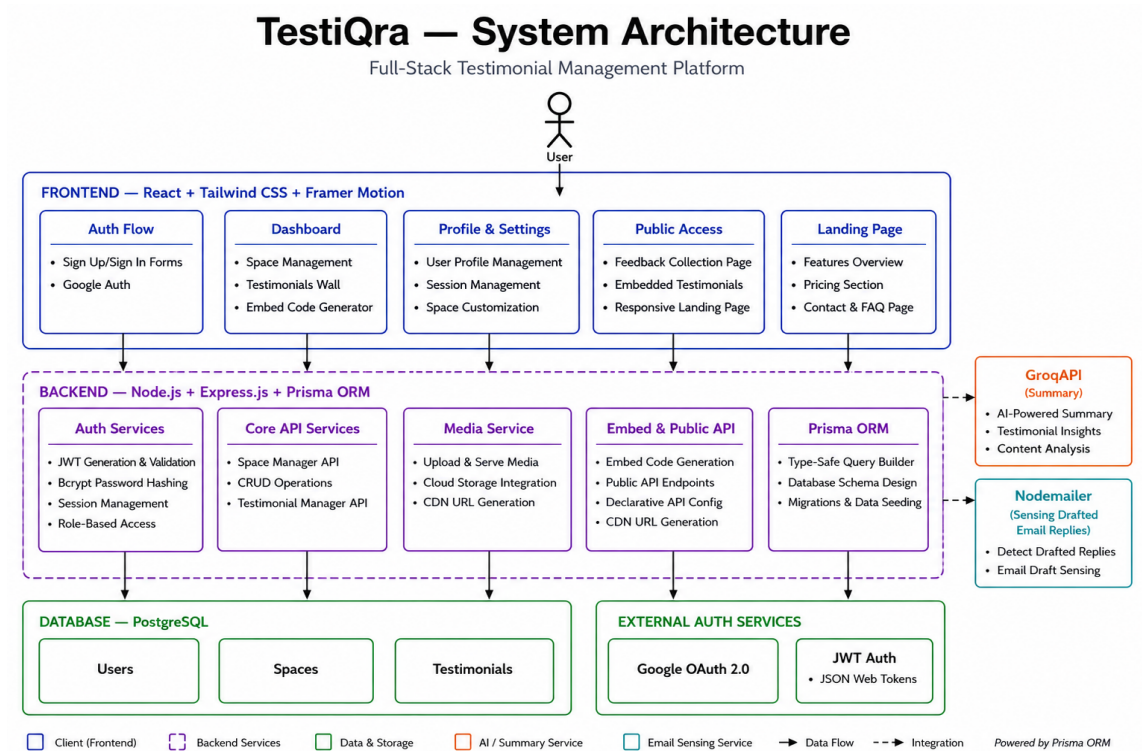


Figure 1: High-Level System Architecture of TestiQra

A2: Entity-Relationship Diagram

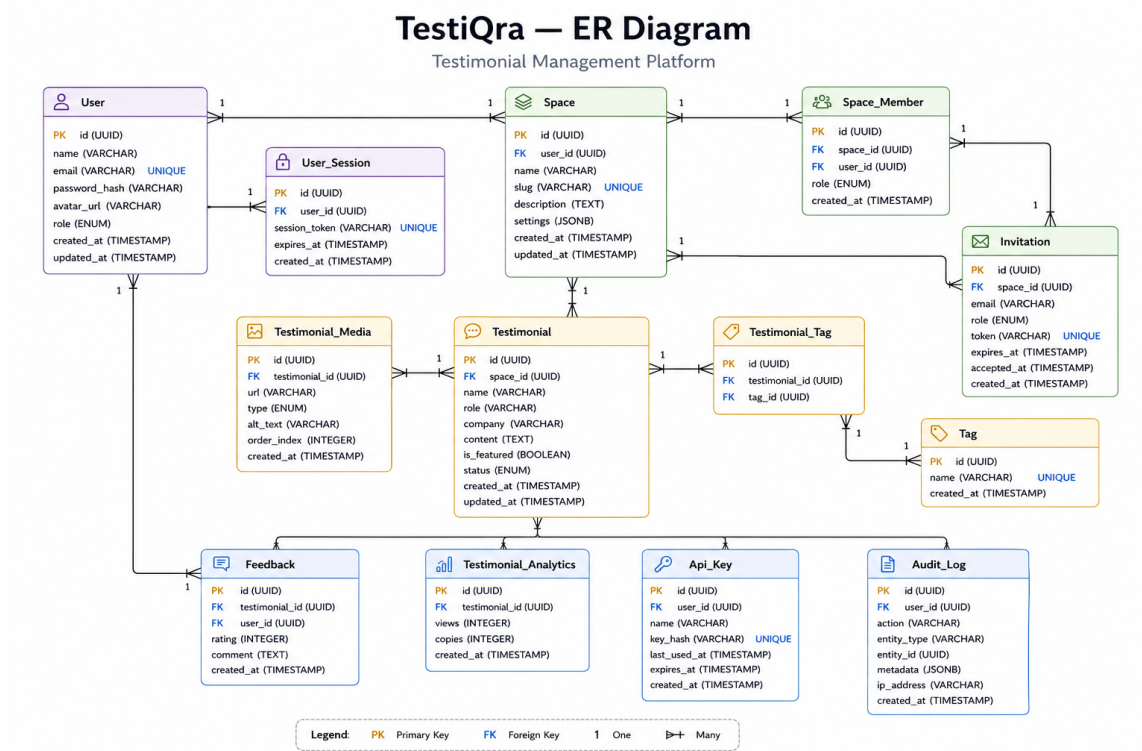


Figure 2: Entity-Relationship Diagram of TestiQra Database

A3: System Flow Diagram

System Flow Diagram — Business Owner Workflow

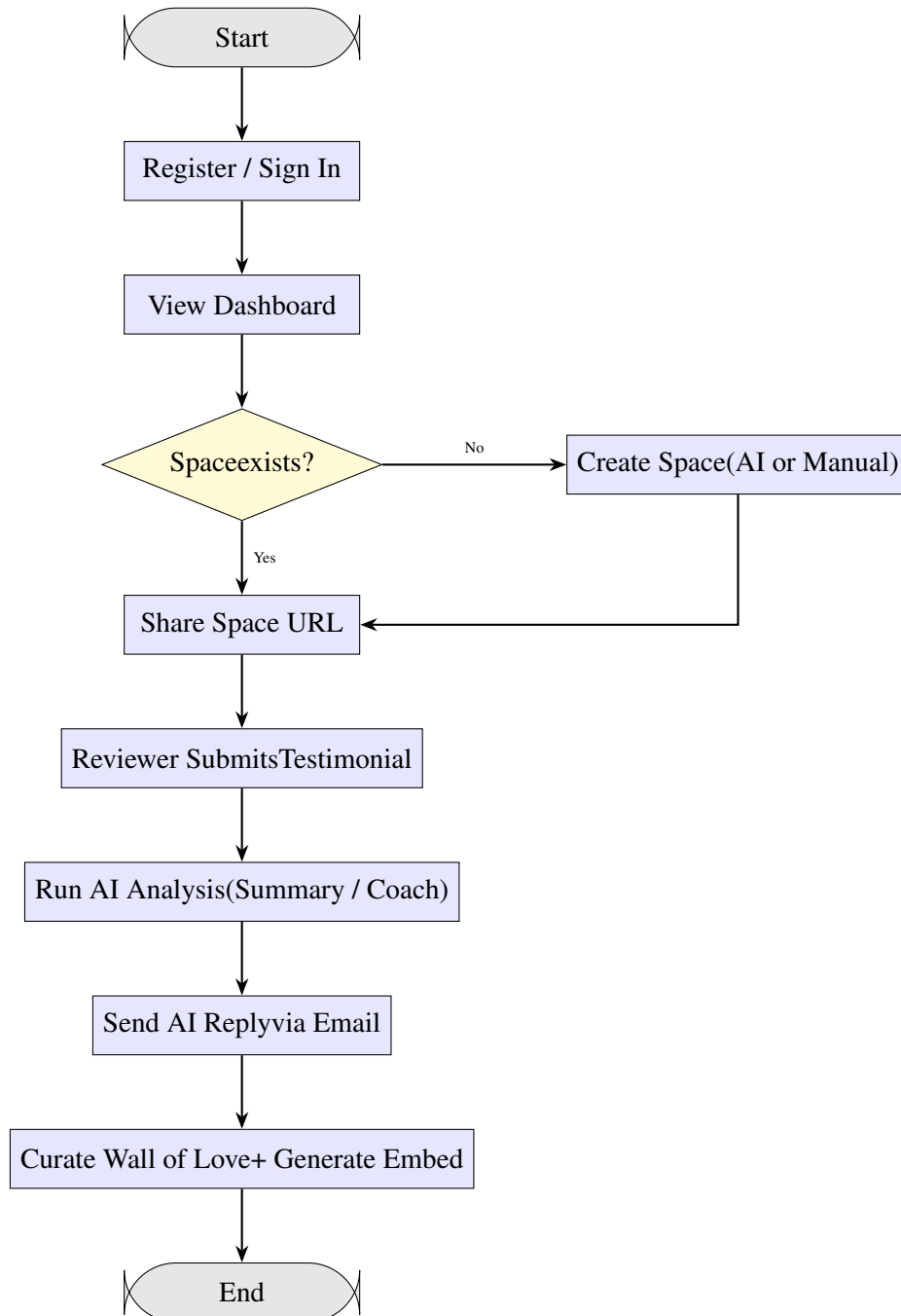


Figure 3: System Flow Diagram — Business Owner Workflow

Pasted content 9.25 KB • 281 lines • Formatting may be inconsistent from source

A5: Program Codes

A5.1 — JWT Authentication Middleware

The following listing presents the complete JWT authentication middleware responsible for protecting all management API endpoints. Every protected Express route passes through this function before the domain-specific handler is invoked.

```
1 import jwt from "jsonwebtoken";
2 import JWT_SECRET from "./config.js";
3
4 const AuthMiddleware = (req, res, next) => {
5   try {
6     const authHeader = req.headers.authorization;
7     if (!authHeader || !authHeader.startsWith("Bearer ")) {
8       return res.status(401).json({ error: "Authorization required"
9       });
10    }
11    const token = authHeader.split(" ")[1];
12    const decoded = jwt.verify(token, JWT_SECRET);
13    if (!decoded.email || !decoded.firstname) {
14      return res.status(401).json({ error: "Invalid token" });
15    }
16    req.email = decoded.email;
17    req.firstname = decoded.firstname;
18    next();
19  } catch (error) {
20    if (error instanceof jwt.TokenExpiredError) {
21      return res.status(401).json({ error: "Token expired" });
22    }
23    return res.status(401).json({ error: "Invalid token" });
24  }
25 }
26 export default AuthMiddleware;
```

Listing 1: JWT Authentication Middleware (middleware.js)

A5.2 — Groq AI Router

The following listing presents the AI feature proxy router. All five AI capabilities are dispatched through this single endpoint, with prompt composition handled on the front-end before dispatch.

```

1 import Groq from "groq-sdk";
2 const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
3
4 AiRouter.post("/", async (req, res) => {
5   const { prompt } = req.body;
6   const completion = await groq.chat.completions.create({
7     model: "llama-3.3-70b-versatile",
8     messages: [
9       {
10        role: "system",
11        content: "You are a JSON-only response bot. " +
12                "Always respond with valid JSON. No markdown."
13      },
14      { role: "user", content: prompt },
15    ],
16    max_tokens: 1024,
17    temperature: 0.4,
18    response_format: { type: "json_object" },
19  });
20  const result = completion.choices?.[0]?.message?.content;
21  const cleaned = result.replace(/'json|'"/g, "").trim();
22  res.json({ result: cleaned });
23 });

```

Listing 2: Groq AI Router (router/AiRouter.js)

A5.3 — Multi-Layer Spam Prevention Pipeline

The following listing presents the four-layer spam prevention pipeline applied to every incoming public testimonial submission request.

```

1 // Layer 1      Honeypot (silent accept to prevent bot adaptation)
2 if (req.body._gotcha || req.body.website) {
3   return res.status(200).json({ message: "testimonial has been
4     sent" });
5 }
6 // Layer 2      IP rate limiting (max 10 / hour; block after 30
7     total)
8 const ip = req.headers['x-forwarded-for']?.split(',')?.[0]?.trim()
9   || req.socket.remoteAddress || 'unknown';
10 if (blockedIPs.has(ip)) {
11   return res.status(429).json({ message: "Too many requests." });
12 }
13 if (isRateLimited(ipSubmissions, ip, 10, 60*60*1000)) {

```

```

12  const hist = ipSubmissions.get(ip) || [];
13  if (hist.length >= 30) blockedIPs.add(ip);
14  return res.status(429).json({ message: "Too many requests." });
15 }
16 // Layer 3      Email rate limiting (max 3 per unique email / 24
    hours)
17 const email = req.body.testimonial?.email?.toLowerCase().trim();
18 if (isRateLimited(emailSubmissions, email, 3, 24*60*60*1000)) {
19   return res.status(429).json({
20     message: "You have already submitted recently." });
21 }
22 // Layer 4      Content pattern filtering (URL, keyword, repetition
    )
23 const looksLikeSpam = (text = '') =>
24   [/https?:\/\/\/gi,
25    /\b(viagra|casino|bitcoin|lottery|prize)\b/gi,
26    /(.)\1{6,}/].some(p => p.test(text));
27 if (looksLikeSpam(req.body.testimonial?.content)) {
28   return res.status(400).json({ message: "Flagged as spam." });
29 }

```

Listing 3: Multi-Layer Spam Prevention (router/TestimonialRouter.js)

A5.4 — Complete Prisma Schema

The following listing presents the complete Prisma schema file defining the TestiQra database structure, including all models, fields, relations, and constraints.

```

1  generator client {
2    provider = "prisma-client-js"
3  }
4
5  datasource db {
6    provider = "postgresql"
7    url      = env("DATABASE_URL")
8  }
9
10 model User {
11   id          String    @id @default(uuid())
12   firstName   String
13   email       String    @unique
14   password    String?
15   googleId    String?   @unique
16   authProvider String    @default("local")

```



```

17   createdAt      DateTime @default(now())
18   updatedAt      DateTime @updatedAt
19   spaces         Space []
20 }
21
22 model Space {
23   id              String      @id @default(uuid())
24   space_name      String      @unique
25   logo            String?
26   header          String
27   customMessage   String?
28   userId          String
29   hide_gif        Boolean      @default(false)
30   thankyou_img_url String
31   thankyou_msg     String
32   thankyou_title   String
33   redirectPageUrl  String
34   user            User         @relation(fields: [userId],
35                                     references: [id])
36   testimonials     Testimonial[]
37   questions        question[]
38 }
39
40 model question {
41   id              String @id @default(uuid())
42   spaceId         String
43   space           Space  @relation(fields: [spaceId], references: [id])
44   question        String
45 }
46
47 model Testimonial {
48   id              String      @id @default(uuid())
49   username        String
50   email           String
51   isTextContent   Boolean
52   Content         String
53   imageURL        String
54   UserImageURL    String
55   Rating          Int
56   videoUrl        String      @default("")
57   space           Space      @relation(fields: [spaceId], references:
58   [id])
58   spaceId         String

```

```

59   submittedAt    DateTime @default(now())
60   liked          Boolean  @default(false)
61 }

```

Listing 4: Complete Prisma Schema (prisma/schema.prisma)

A5.5 — AI Prompt Templates

The following listings present all five AI prompt templates used by TestiQra. Each template is composed on the front-end with runtime values substituted for the placeholder variables before dispatch to the AI router.

Space Content Generation:

```

Generate testimonial collection content for:
Company: {name}, Type: {type},
Target Audience: {audience}, Product/Service: {product}
Return ONLY valid JSON:
{
  "header": "compelling header under 35 characters",
  "customMessage": "warm, encouraging 2-3 sentence welcome message",
  "questions": ["question1","question2","question3","question4","question5"]
}

```

AI Sentiment Summary:

```

You are a business analyst. Analyse the following testimonials.
Return ONLY valid JSON:
{
  "overallSentiment": "Positive|Mixed|Negative",
  "sentimentScore": <integer 0-100>,
  "keyPositives": ["theme1","theme2","theme3"],
  "keyComplaints": ["issue1","issue2","issue3"],
  "summary": "2-3 sentence executive summary paragraph."
}
Testimonials: {testimonials_corpus}

```

Improvement Coaching:

```

You are a business improvement consultant. Identify recurring
  issues.
Return ONLY valid JSON:
{
  "issues": [

```

```
{
  "theme": "issue name",
  "priority": "High|Medium|Low",
  "description": "what customers say",
  "action": "specific, concrete remediation action"
}
]
}
Testimonials: {testimonials_corpus}
```

AI Reply Drafting:

```
Write a SHORT, warm, genuine reply email. 2-3 sentences maximum.
No hashtags. No emojis. Reference their experience specifically.
Return ONLY valid JSON: {"reply": "your reply text here"}
Customer: {username}, Rating: {rating}/5
Testimonial: "{content}"
```

Testimonial Curation Recommendation:

```
You are a marketing strategist. Select the best testimonials for a
Wall of Love gallery that maximises social proof impact.
Return ONLY valid JSON:
{
  "recommended_ids": ["id1", "id2", "id3"],
  "reasoning": "brief explanation of selection criteria"
}
Testimonials (with IDs): {testimonials_with_ids}
```

A5.6 — Environment Variable Reference

Table 1: Required Environment Variables

Variable	Description	Tier
DATABASE_URL	PostgreSQL connection string (Prisma)	Server
JWT_SECRET	Secret key for signing JWTs	Server
GROQ_API_KEY	Groq API authentication key	Server
GOOGLE_CLIENT_ID	Google OAuth 2.0 client ID	Server + Client
EMAIL_USER	Gmail account for SMTP	Server
EMAIL_PASS	Gmail App Password	Server
VITE_BACKEND_URL	Base URL of the Express API	Client (Vite)
VITE_FRONTEND_URL	Base URL of the React SPA	Client (Vite)
CLOUDINARY_CLOUD_NAME	Cloudinary cloud account name	Server
CLOUDINARY_API_KEY	Cloudinary API key	Server
CLOUDINARY_API_SECRET	Cloudinary API secret	Server

A5.7 — Space Creation Router

The following listing presents the Express router that handles the creation of a new testimonial-collection *Space*. The handler validates the incoming payload with Zod, resolves the owner from the JWT-decoded e-mail, persists the space record, and then bulk-creates the associated questions in a loop.

```
1 import express from "express";
2 import z from "zod";
3 import Authmiddleware from "../middleware.js";
4 import { PrismaClient } from "@prisma/client";
5
6 const prisma = new PrismaClient();
7 const SpaceCreationRouter = express.Router();
8
9 const SpaceSchema = z.object({
10   spacename: z.string(),
11   imageUrl: z.string().optional(),
12   header: z.string(),
13   customMessage: z.string(),
14   questions: z.array(z.string()),
15   hideImage: z.boolean(),
16   redirect_url: z.string(),
17   imagePreview: z.string().optional(),
18   thankyouTitle: z.string(),
```

```

19   thankyouMessage: z.string(),
20 });
21
22 SpaceCreationRouter.post("/", Authmiddleware, async (req, res) => {
23   try {
24     const validationResult = SpaceSchema.safeParse(req.body);
25     if (!validationResult.success)
26       return res.status(400).json({ error: validationResult.error
27     });
28
29     const { spacename, imageUrl, header, customMessage, questions,
30           hideImage, redirect_url, imagePreview,
31           thankyouTitle, thankyouMessage } = req.body;
32
33     const user = await prisma.user.findUnique({
34       where: { email: req.email, firstName: req.firstname },
35     });
36
37     if (user) {
38       const space = await prisma.space.create({
39         data: {
40           userId: user.id,
41           space_name: spacename,
42           logo: imageUrl,
43           header,
44           customMessage,
45           hide_gif: hideImage,
46           redirectPageUrl: redirect_url,
47           thankyou_img_url: imagePreview,
48           thankyou_title: thankyouTitle,
49           thankyou_msg: thankyouMessage,
50         },
51       });
52
53       for (const question of questions) {
54         await prisma.question.create({
55           data: { spaceId: space.id, question },
56         });
57       }
58
59       res.status(200).json({ message: "space created successfully"
60     });
61   }
62 }

```

```

60   } catch (err) {
61     console.error(err);
62     res.status(500).json({ error: "Internal server error" });
63   }
64 });
65
66 export default SpaceCreationRouter;

```

Listing 5: Space Creation Router (router/SpaceCreation.js)

A5.8 — Edit Space Router

The following listing presents the PUT `/api/v1/edit` endpoint. After schema validation, the handler locates the space by its current slug, updates all mutable fields in one `prisma.space.update` call, replaces every existing question by deleting the old set and re-creating entries from the request array, then returns the updated space and question list.

```

1  import express from "express";
2  import z from "zod";
3  import { PrismaClient } from "@prisma/client";
4  import Authmiddleware from "../middleware.js";
5
6  const prisma = new PrismaClient();
7  const editRouter = express.Router();
8
9  const SpaceSchema = z.object({
10    spacename:      z.string(),
11    imageUrl:       z.string().optional(),
12    header:         z.string(),
13    customMessage:  z.string(),
14    questions:      z.array(z.string()),
15    hideImage:      z.boolean(),
16    redirect_url:    z.string(),
17    imagePreview:    z.string().optional(),
18    thankyouTitle:   z.string(),
19    thankyouMessage: z.string(),
20  });
21
22 editRouter.put("/", Authmiddleware, async (req, res) => {
23   try {
24     const { spaceName } = req.query;           // current slug
25     from URL
26     const valid = SpaceSchema.safeParse(req.body);
27     if (!valid.success)

```

```

27     return res.status(400).json({ error: valid.error });
28
29     const existing = await prisma.space.findUnique({
30       where: { space_name: spaceName },
31     });
32     if (!existing)
33       return res.status(404).json({ error: "Space not found" });
34
35     const { spacename, imageUrl, header, customMessage, questions,
36           hideImage, redirect_url, imagePreview,
37           thankyouTitle, thankyouMessage } = req.body;
38
39     const updated = await prisma.space.update({
40       where: { space_name: spaceName },
41       data: {
42         space_name:      spacename,
43         logo:            imageUrl,
44         header,
45         customMessage,
46         hide_gif:        hideImage,
47         redirectPageUrl: redirect_url,
48         thankyou_img_url: imagePreview,
49         thankyou_title:  thankyouTitle,
50         thankyou_msg:    thankyouMessage,
51       },
52     });
53
54     // Replace questions atomically
55     await prisma.question.deleteMany({ where: { spaceId: updated.id } });
56     const updatedQs = await Promise.all(
57       questions.map(q =>
58         prisma.question.create({ data: { spaceId: updated.id,
59           question: q } })
60       )
61     );
62
63     res.status(200).json({
64       message: "Space updated successfully",
65       space:   updated,
66       questions: updatedQs,
67     });
68   } catch (err) {

```

```

68     res.status(500).json({ error: "Internal server error",
69                           details: err.message });
70   }
71 });
72
73 export default editRouter;

```

Listing 6: Edit Space Router (router/editspace.js)

A5.9 — Delete Space Router

The following listing presents the DELETE `/api/v1/space/:space_name` endpoint. Because Prisma enforces foreign-key constraints, dependent question and testimonial rows must be removed before the space record itself can be deleted. The three deletions are executed in the required order inside a single try block.

```

1  import express from "express";
2  import { PrismaClient } from "@prisma/client";
3  import Authmiddleware from "../middleware.js";
4
5  const prisma = new PrismaClient();
6  const DeleteSpaceRouter = express.Router();
7
8  DeleteSpaceRouter.delete("/:space_name", Authmiddleware, async (req
9    , res) => {
10    try {
11      const { space_name } = req.params;
12
13      const space = await prisma.space.findUnique({ where: {
14        space_name } });
15      if (!space)
16        return res.status(404).json({ error: "Space not found" });
17
18      // Remove child records first (FK constraints)
19      await prisma.question.deleteMany({ where: { spaceId: space.id
20        } });
21      await prisma.testimonial.deleteMany({ where: { spaceId: space.
22        id } });
23      await prisma.space.delete({ where: { space_name } });
24
25      res.status(200).json({ message: "Space deleted successfully"
26        });
27    } catch (err) {
28      res.status(500).json({ error: "Internal server error",

```

```
24         details: err.message });  
25     }  
26 });  
27  
28 export default DeleteSpaceRouter;
```

Listing 7: Delete Space Router (router/DeleteSpaceRouter.js)

A6: Application Screenshots

Sample Application Screenshots

The following figures present annotated screenshots from the fully implemented TestiQra application, demonstrating the primary user interface views and AI feature outputs.

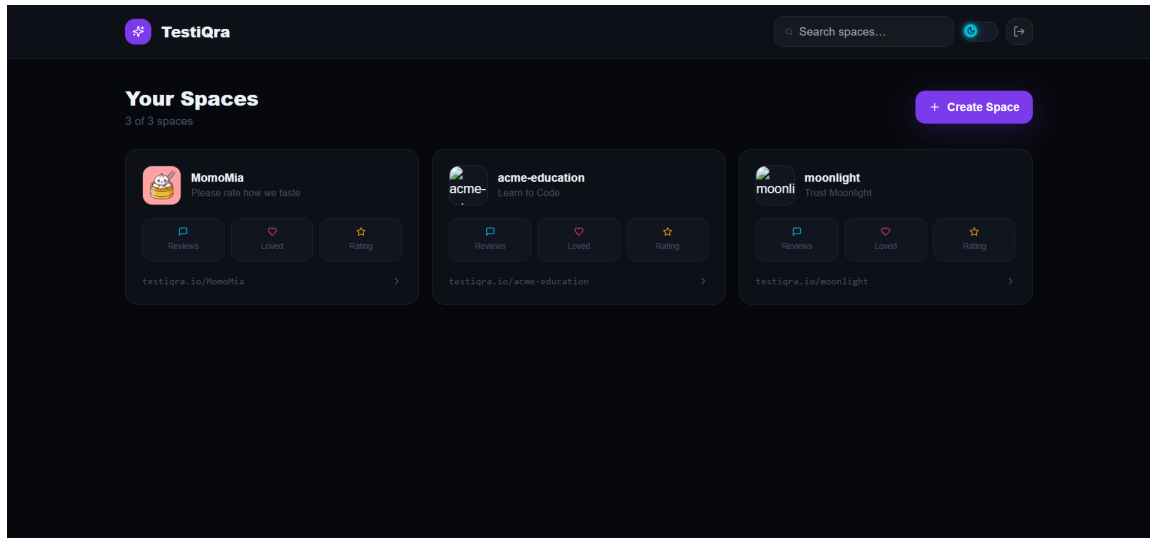


Figure 4: TestiQra Dashboard — Space Management View showing Space cards with quick-action buttons, search bar, and dark/light theme toggle.

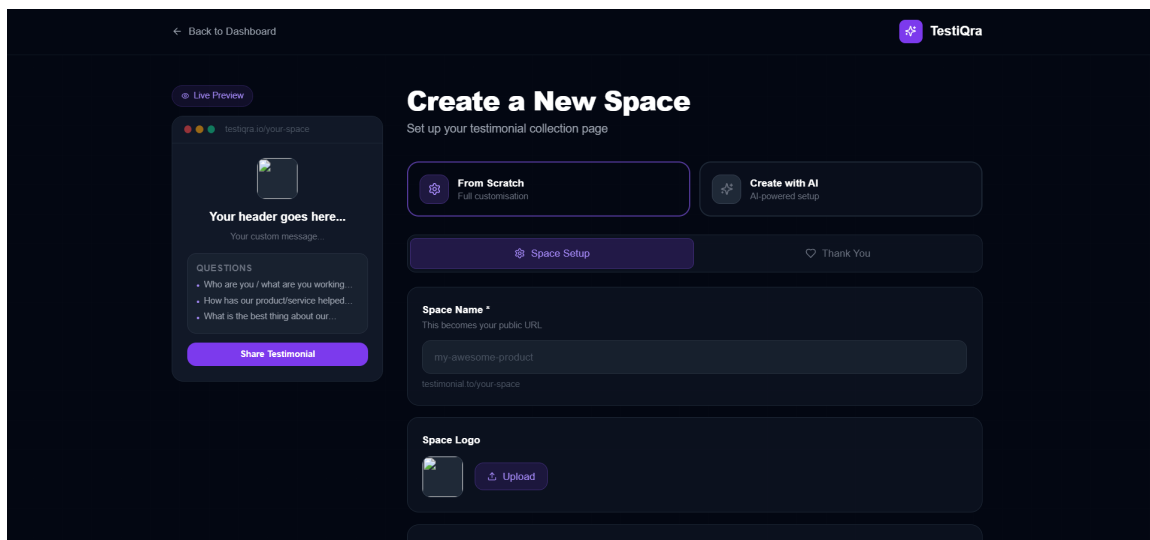


Figure 5: Space Creation Interface — From Scratch mode showing all configuration fields, logo upload, and question management panel.

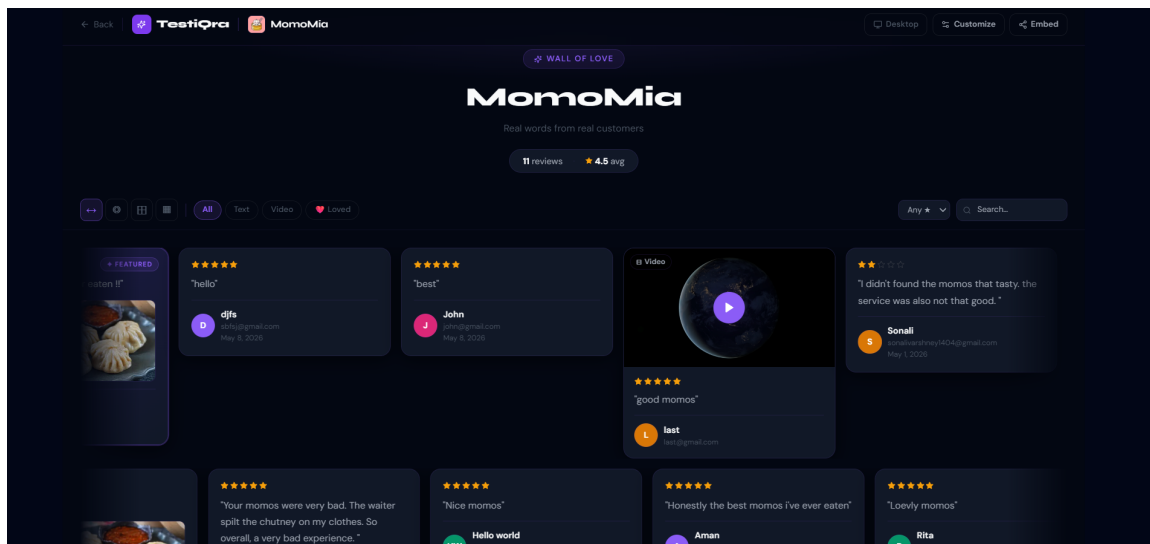


Figure 6: Wall of Love — Embeddable gallery with real-time theme and layout customiser panel open, showing live preview before embed code generation.

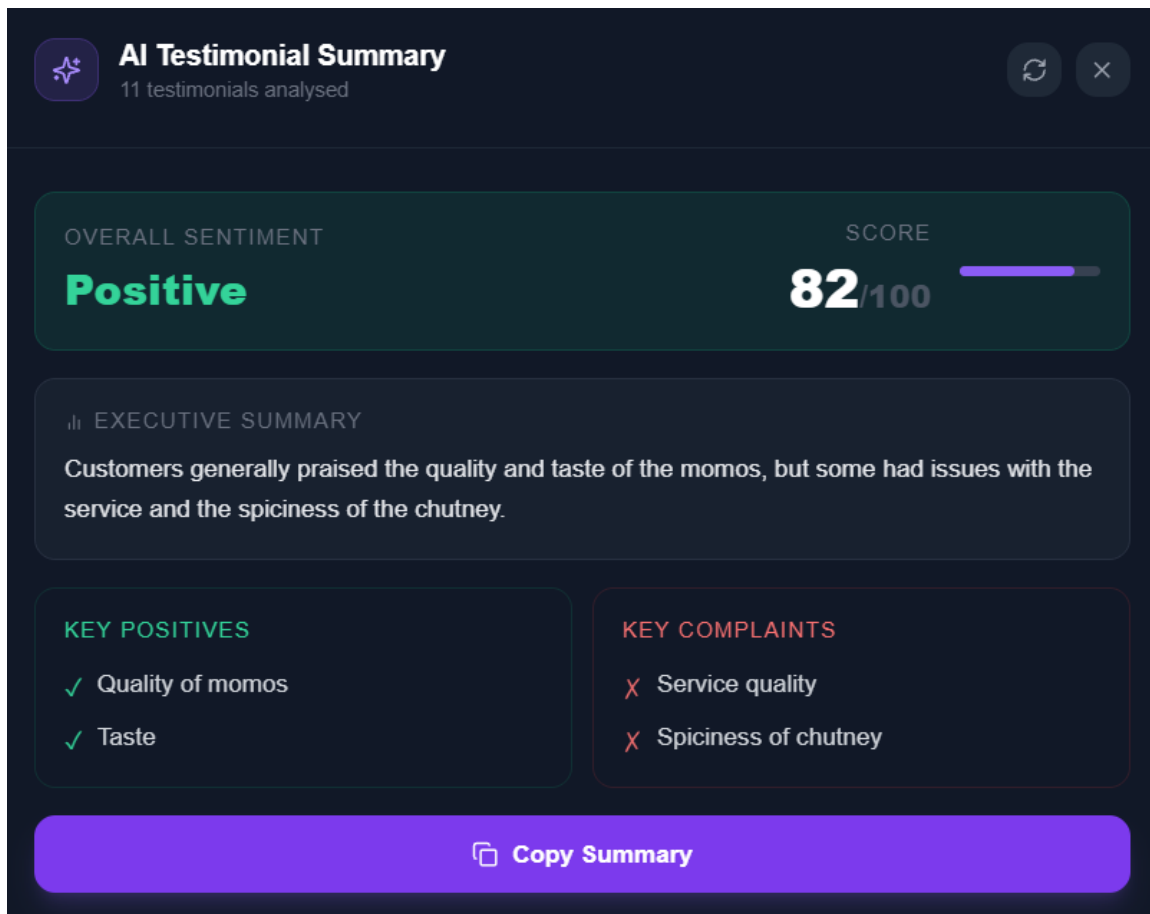


Figure 7: AI Sentiment Summary Output — Executive dashboard view showing sentiment score, key positives, key complaints, and prose summary card.



Learn to Code

We're proud of our students' success! If Acme Education's programming courses have helped you achieve your goals, we'd love to hear about it. Share your story with us and inspire others to start their learning journey.

ANSWER THESE QUESTIONS

- ◆ What motivated you to enroll in our programming courses?
- ◆ How has Acme Education's programming courses helped you improve your skills?
- ◆ What was your favorite part of the course and why?
- ◆ How has the knowledge you gained from our courses impacted your career or projects?
- ◆ Would you recommend Acme Education's programming courses to others and why?

 Share your testimonial >

◆ Includes text & video options

Powered by  TestiQra

Figure 8: Public Testimonial Submission Form — Reviewer-facing interface showing star rating selector, text/video toggle, and profile photo upload.

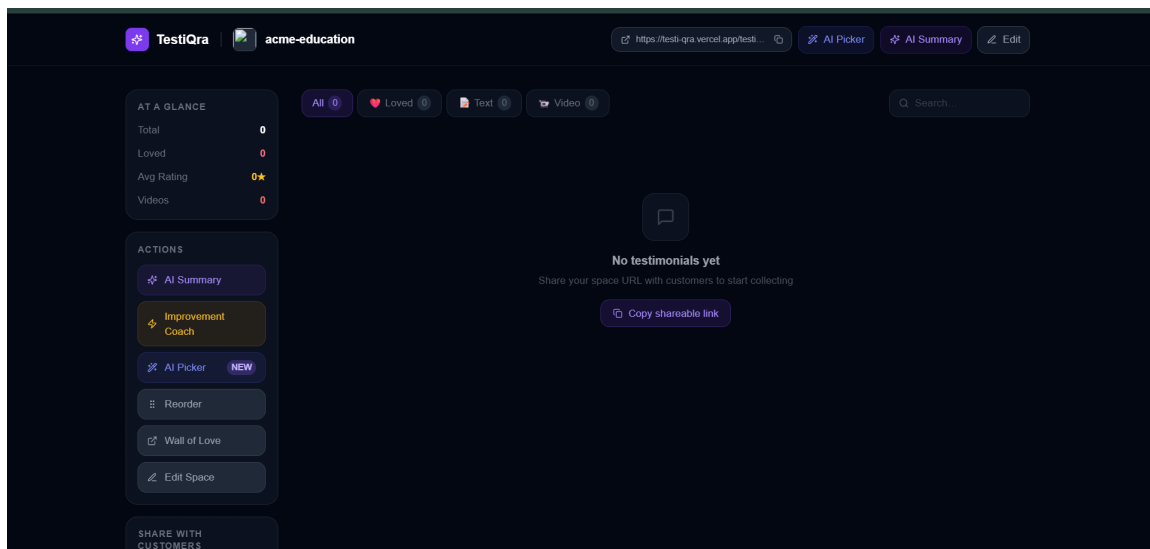


Figure 9: Space Dashboard View — Testimonial management interface showing testimonial cards with like, reply, and filter controls.